Meeting 13. Solving Linear Systems

The walk · Lines, planes, and the condition number · Elimination, three ways · Residual and error · Sparse matrices · Fill-in · Solving by iterating

A **linear system** is a set of equations that must hold together, each a sum of unknowns multiplied by numbers. Meeting 13 is about what such a system looks like, what it costs to solve, how far a computed solution can be trusted, and what changes when the matrix is mostly zeros. It opened with a question that is not obviously about equations at all: a particle wanders at random over a graph; from a given node, how often does it reach the red node before the blue one?

Every group did two notebooks before the meeting of Wednesday, October 14: the first on linear systems and their pictures, ending with the walk; the second on three ways to solve, round-off, and sparse matrices. Each student uploaded both, and each group posted twice on the board. The meeting itself was taught from three web pages with moving pictures, not from slides, and with a little whiteboard for one count: the walk simulated and then answered as a linear system; lines and planes, nearly parallel lines and the condition number, elimination as planes that pivot; the three solvers racing; the residual against the error; sparse storage and why zeros are neutral; fill-in and the order of elimination; and solving by iterating, by projection onto one equation at a time. These notes state the main points first, then walk through both notebooks part by part, then develop fill-in and iteration. The boxes *In the meeting* are from the transcript. The notebooks were run for these notes on a machine with 32 logical cores, so its timings are faster than Colab's; the class's own Colab numbers are quoted where it matters.

The notebooks, the pages and the decks of this meeting.

Each notebook opens in Google Colab. Choose *Save a copy in Drive* in the *File* menu before you change anything, to save your work.

- The first notebook: linear systems (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-13-linear-systems.ipynb)
  Parts 1 to 4: two unknowns and two lines, three unknowns and three planes, nearly parallel lines and the condition number, a walk on a graph.
- The second notebook: solvers and sparse matrices (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-13-solvers.ipynb)
  Parts 1 to 6: elimination with loops, three ways to solve, round-off and the condition number, the formats COO, CSR and CSC, a grid solved dense and sparse, the product as the matrix fills.

The three pages the meeting was taught from; each runs in the browser, and the controls do what they say.

- A walk on a graph (https://ikoutis.github.io/cs301-labs/unit-2/pages/random-walk.html) the walk of Part 4 on the notebook's graph, one walker or a thousand, and the exact answer behind a button
- Linear systems, in pictures (https://ikoutis.github.io/cs301-labs/unit-2/pages/linear-systems.html) two lines, nearly parallel lines, three planes that turn, and elimination as planes that pivot
- Solvers, round-off and sparse matrices, in motion (https://ikoutis.github.io/cs301-labs/unit-2/pages/solvers-and-sparse.html)
  the race, the residual against the error, the formats, fill-in, and Kaczmarz, in the plane and in space

The four decks, which cover the same ground slide by slide, with the narration in their notes; they were not shown in the meeting.

- Lines, Planes, and a Walk (https://ikoutis.github.io/cs301-labs/unit-2/slides/m13-recap-1.html) Recap 1 (PDF)
- Solvers, Round-off, and Sparse Matrices (https://ikoutis.github.io/cs301-labs/unit-2/slides/m13-recap-2.html)
  Recap 2 (PDF)
- Elimination and Fill-in (https://ikoutis.github.io/cs301-labs/unit-2/slides/m13-deeper-1.html) A deeper look 1 (PDF)
- Solving by Iterating (https://ikoutis.github.io/cs301-labs/unit-2/slides/m13-deeper-2.html) A deeper look 2 (PDF)

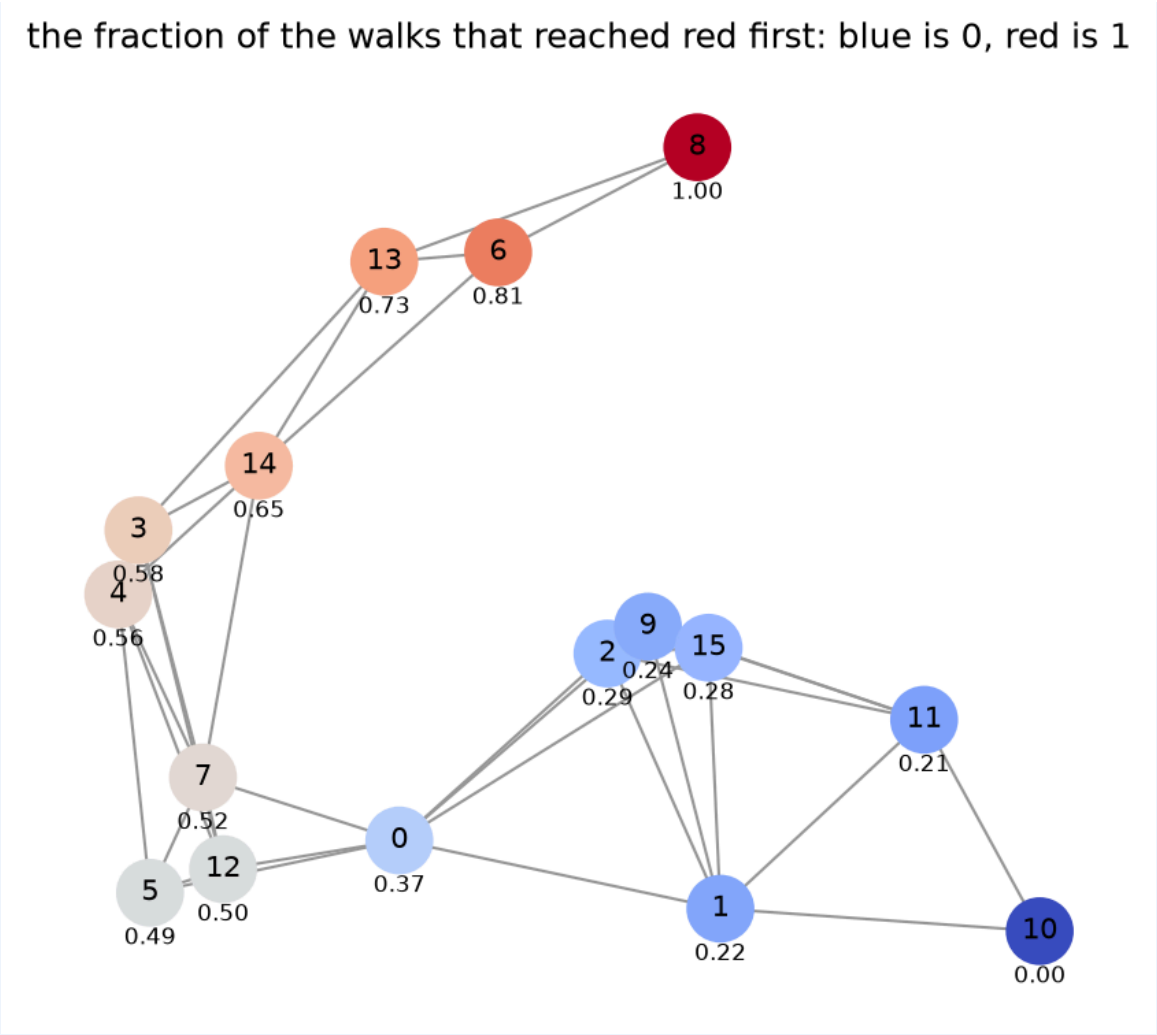
> The notebooks are the reference for Review Quiz 13, whose questions name their parts.

The main points

The following ten ideas summarize the meeting. Each links to the explanation and the code.

1 A walk on a graph is a model, and a simulation answers it slowly

A **graph** is a set of **nodes** joined by **edges**: web pages and their links, people and their friendships. Two nodes of the notebook's graph are marked, one **red** and one **blue**, and a particle starts at some node and moves, at every step, to a neighbour chosen at random, until it reaches red or blue. The model in the meeting was a reader of web pages who follows links at random, and the question was which of two pages, two parties, the reader reaches first from a given start. One walk is a random event; two thousand walks from each node give a *fraction* for each node, which estimates a probability. The fractions wobble as the walks come in, and a few walkers wander for a long time before they are absorbed.



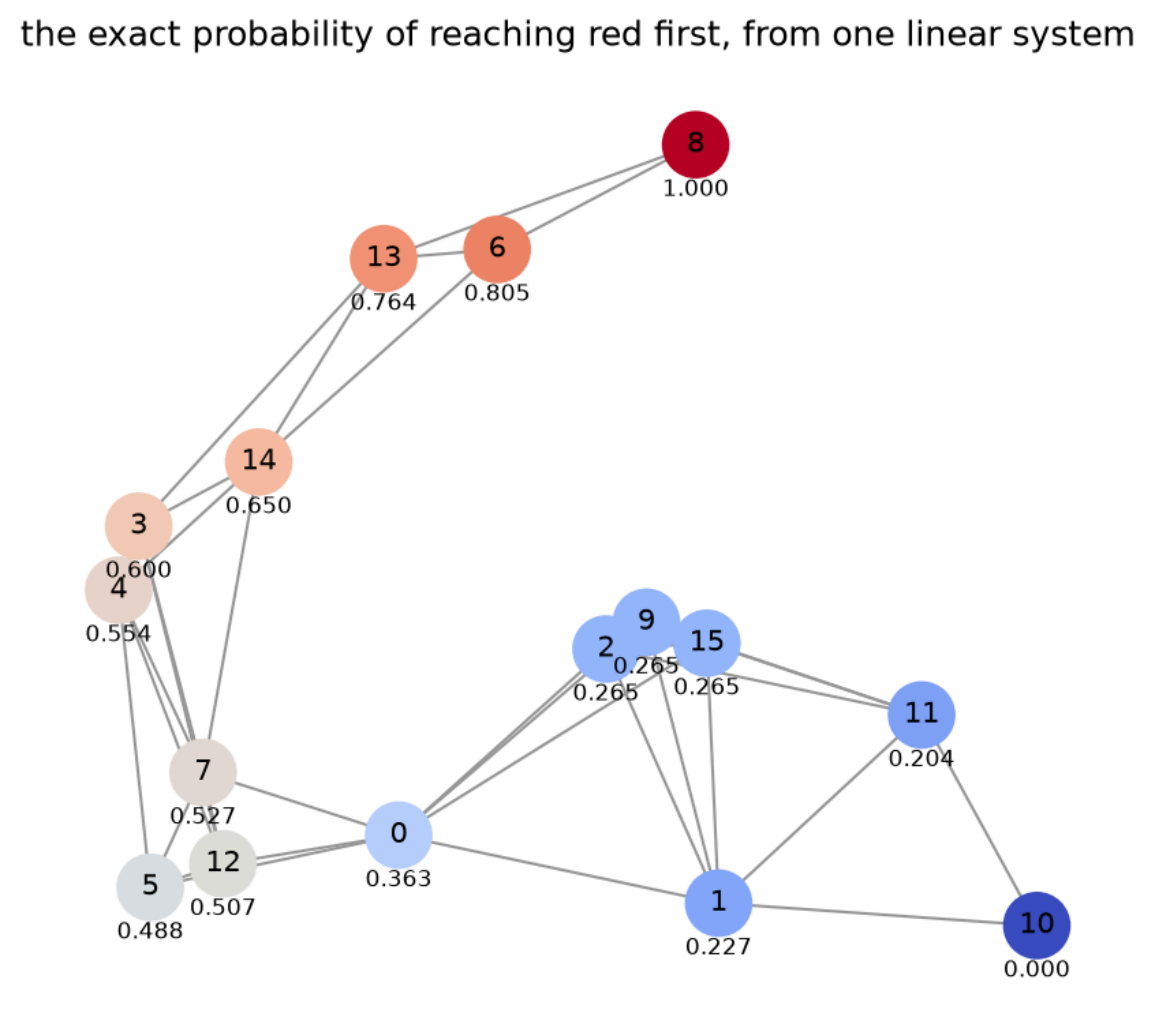
The notebook's graph: red is node 8, blue is node 10. Each node is coloured by the fraction of its 2,000 walks that reached red first.

Section 1.4.

2 The exact answer is a linear system: every node is the average of its neighbours

Call x_v the probability that a walk from node v reaches red first. At red it is 1 and at blue it is 0. From any other node the first step goes to each neighbour with the same probability, and from there the chance is that neighbour's own number: so x_v is the **average of the neighbours' probabilities**. Node 15 has the neighbours 0, 1, 2, 9 and 11, so $x_{15} = (x_0 + x_1 + x_2 + x_9 + x_{11}) / 5$. That is a linear equation, one unknown multiplied by numbers and added; there is one for each of the sixteen nodes, sixteen equations in sixteen unknowns, and `np.linalg.solve` gives all sixteen numbers at once, to every digit the computer keeps. The exact probability of reaching red first from node 0 is 0.363, from node 1 0.227; the largest difference between the 2,000 walks and the exact

solution was 0.032. Solving beats simulating twice over: it is faster, and it is exact, and when the probabilities decide where a budget goes, the difference between 22 and 20 per cent is a decision.



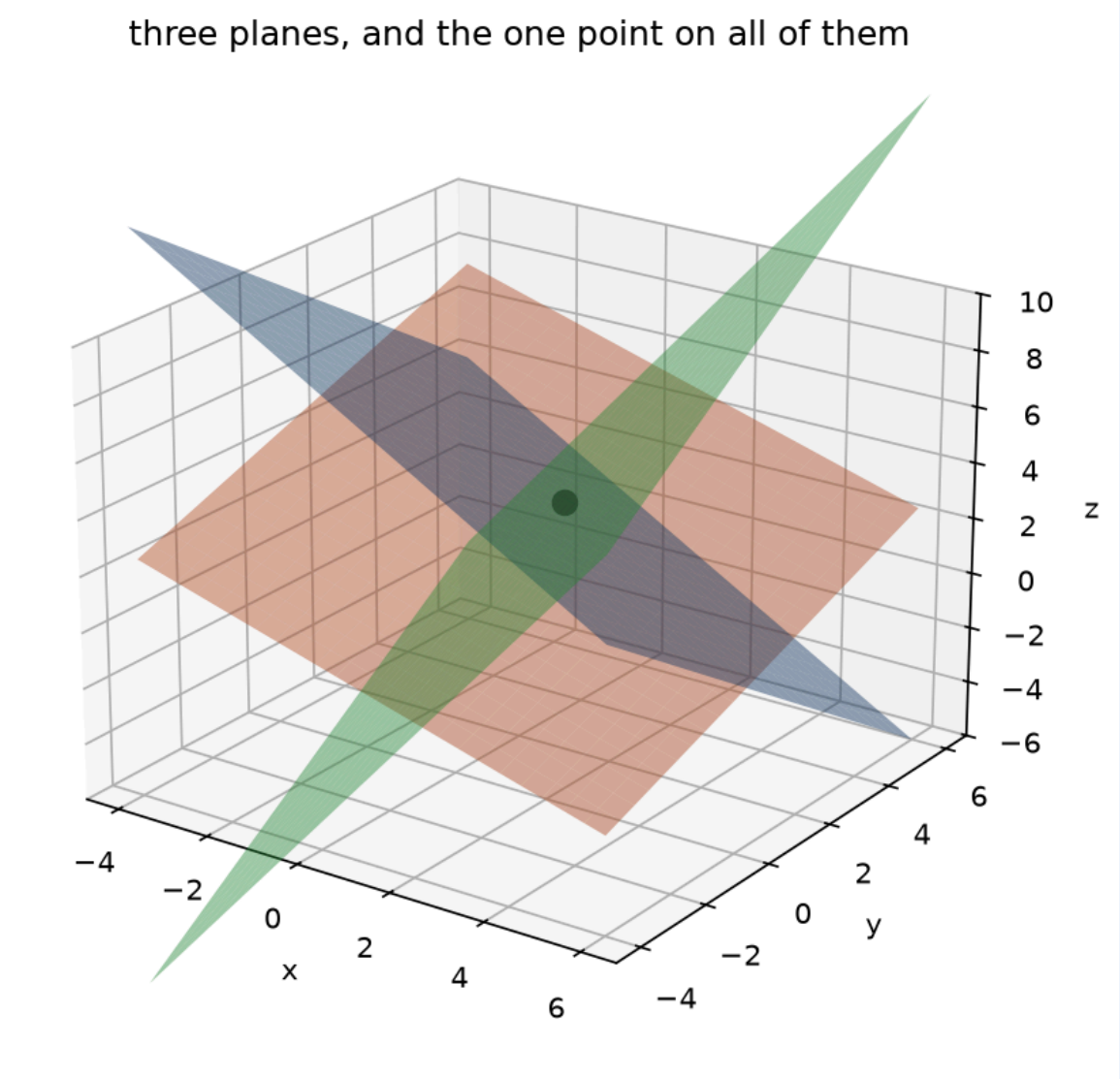
The exact probabilities, from one linear system.

Section 1.4.

3 An equation is a line; a solution is where the lines cross

With two unknowns, one linear equation is satisfied by every point of a **line**; two equations are two lines, and a solution is a point on both, the crossing. With three unknowns each equation is a **plane**, and three planes that are independent meet at one point. With more unknowns the words run out and the objects are called hyperplanes, and the picture is the same: a point where they all meet. In matrix form the rows of `A` are the equations, `A @ x` computes every left-hand side at once, and `np.linalg.solve(A, b)` finds the crossing. The **rank** is the number of equations that are independent; a square system has one solution exactly when its rank is full. When

the third plane contains the line in which the first two meet, every point of that line is a solution; move the plane off the line and no point is on all three; NumPy answers *Singular matrix* in both cases.



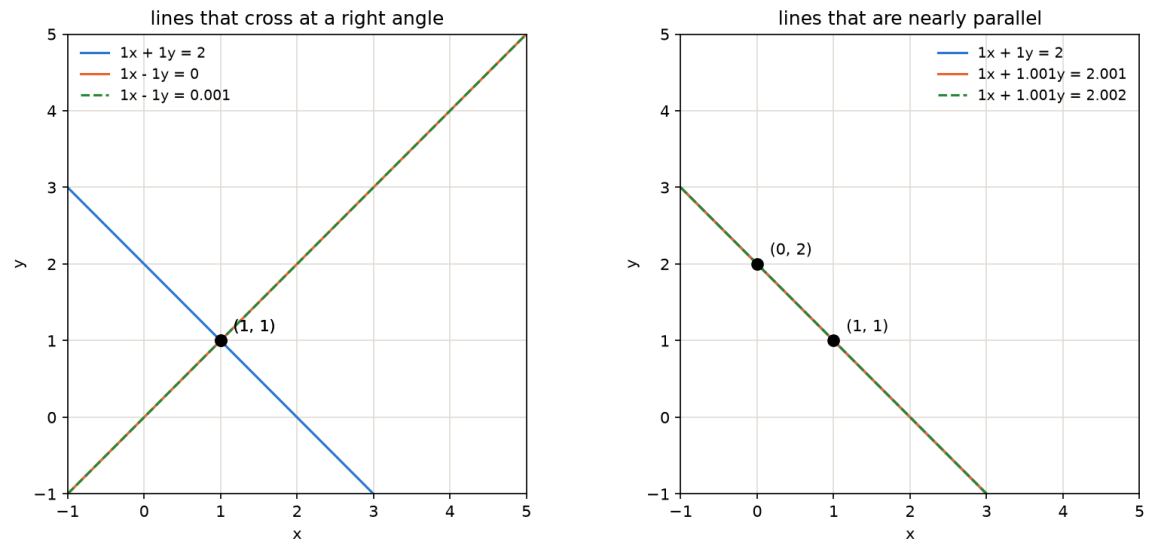
Three planes, and the one point on all of them: x = 1, y = 2, z = 3.

Sections 1.1 and 1.2.

4 Nearly parallel lines: the condition number

Two lines that cross at a right angle show their crossing plainly; two lines that are nearly parallel cross somewhere, but the eye cannot say where, and a change of a thousandth in one equation slides the crossing far along them. The **condition number** of a system, `np.linalg.cond(A)`, measures this: 1 for the right angle, the best there is; large when the lines are nearly parallel; infinite when they are parallel or the same, that is, when the rank drops. An algorithm loses precision where the eye does: with sixteen digits in a

float64 number, a condition number of 10^k can cost up to k of them. In the notebook's pair, $x + y = 2$ and $x + 1.001y = 2.001$, the condition number is about 4,000 and a change of 0.001 in b moves the solution by 1.4.



The same change of a thousandth, at a right angle and at a small angle.

Section 1.3.

5 Elimination pivots the planes; the point stays; the count is about $n^3/3$

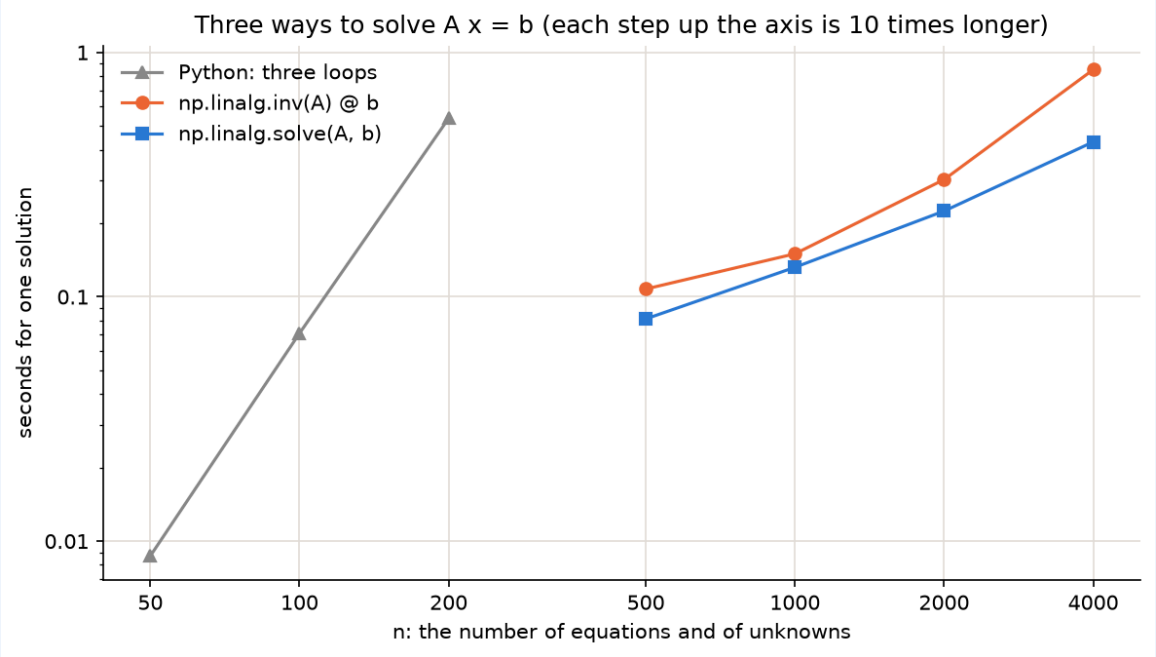
Gaussian elimination subtracts a multiple of one equation from another so that an unknown disappears from it. In the picture, the plane of the changed equation pivots about the line it shares with the other, and the solution does not move; after enough steps the system is triangular, the last equation holds one unknown, and **back substitution** climbs from the last unknown to the first. The count, done on the whiteboard: the first row is subtracted from the $n - 1$ rows below it, n numbers at a time, about n^2 operations; then the next row from the rows below it, about $(n-1)^2$; and so on down to 1^2. The sum of the squares up to n is about $n^3 / 3$: $\mathbf{O(n^3)}$, three loops, and doubling n multiplies the work by about 8.

Section 2.1.

6 Three ways to solve, and the one to use

The loops of Python are hopeless past a few hundred unknowns: on Colab they took 1.4 seconds at n = 200 in the class's posts, which is about three hours at n = 4,000. The inverse, `np.linalg.inv(A) @ b`, does about three times the arithmetic of one elimination and then a product: about 6 seconds at n = 4,000 on Colab. Elimination in compiled

code, `np.linalg.solve(A, b)`, took about 1.5 seconds there. For a system, use `solve`; the inverse is the textbook formula, not the method.

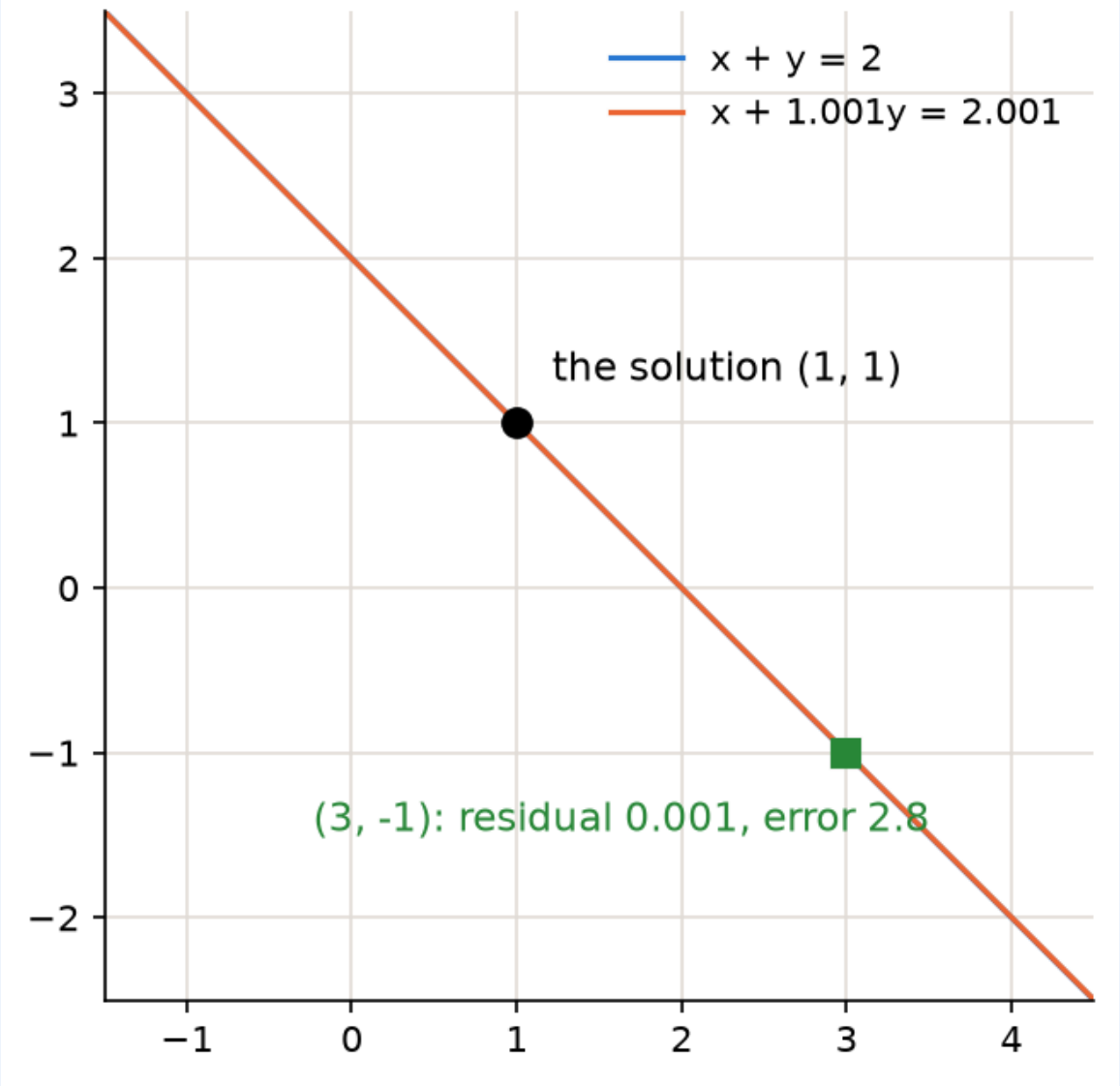


The three ways, timed on the machine that built these notes.

Section 2.2.

7 The residual we can measure; the error we cannot

A solver returns a candidate $\hat{x}$. Its **error**, the distance to the true solution, cannot be computed: if it could, we would add it and be done, and we would not have needed the solver. What can be computed is the **residual**, how far $A\hat{x}$ is from b; it is a proxy for the error, and the only certificate there is. The two can disagree: a candidate can satisfy both equations to within a hair and still be far from the solution, by about the condition number times the residual. A well-made solver returns a residual near 10^{-16} whatever the system; at a condition number of 10^{12} that still leaves about four correct digits, at 10^{16} none.

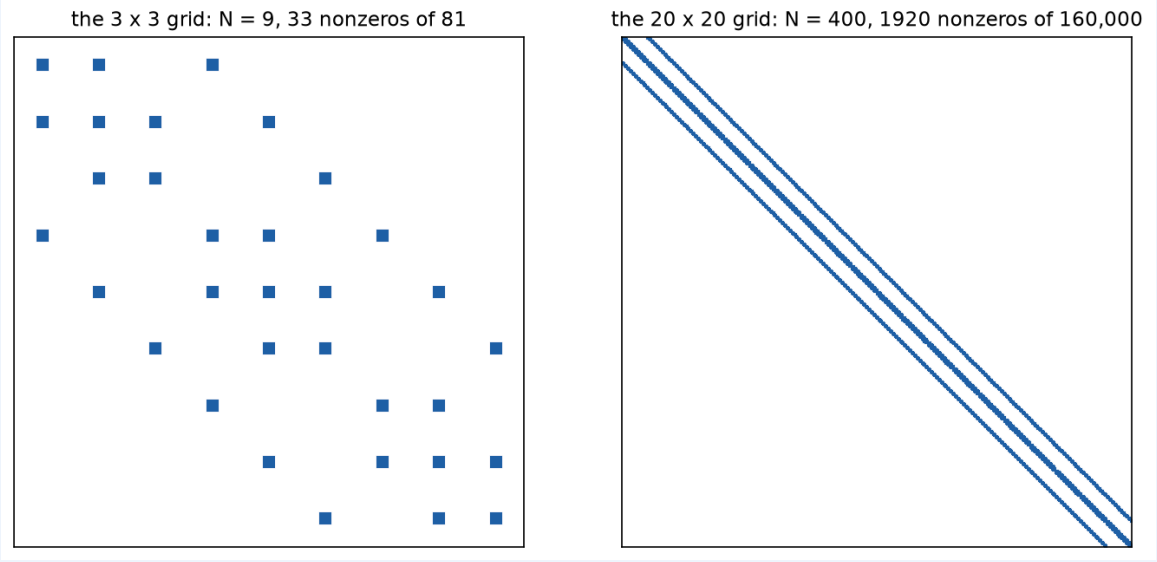


A small residual and a large error, on nearly parallel lines.

Section 2.3.

8 Zeros are neutral: a sparse matrix is stored, and multiplied, without them

The matrix of a graph has a nonzero wherever two nodes are joined and zeros everywhere else; a billion accounts could be joined in a billion squared ways and are joined in a few hundred each, so the matrix is almost all zeros and no computer could store it dense. A **sparse** format keeps the nonzeros and where they are: **COO**, the row, column and value of each; **CSR**, the values and their columns row by row, with an array that says where each row begins; **CSC**, the same by columns. In a product `A @ v` a zero contributes $0 \times v_j = 0$, so the dense product does, and reads, work that changes nothing, and the sparse one skips it: one multiplication for each stored nonzero, $\mathbf{O(nnz)}$. Storage and work both follow the nonzeros, not the size.

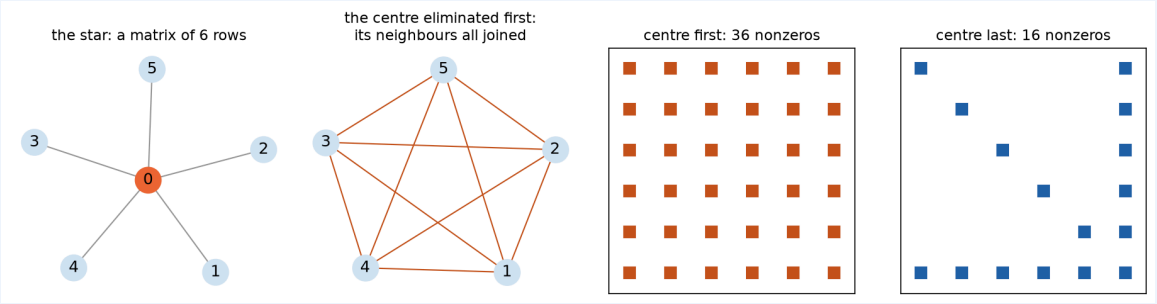


Where the nonzeros of a grid's matrix are: five diagonals, and white everywhere else.

Sections 2.4 to 2.6.

9 Eliminating a node joins its neighbours: fill-in, and the order of elimination

In the matrix of a graph, eliminating unknown k subtracts its row from every neighbour's row, and afterwards every pair of its neighbours is joined: new nonzeros, **fill-in**, that elimination must store and work on. Eliminate the centre of a star first and its leaves become a clique, n nonzeros become about n^2; eliminate the leaves first and nothing is created. Choosing at each step the node with the fewest neighbours is the **minimum degree** order; on the walk's graph it created no fill at all. On a grid the natural order fills a band; **nested dissection** cuts the grid by a separator, eliminates the two halves first and the separator last, and cuts the halves again, so that nothing in one half can ever fill the other. On the 6×6 grid of the page, the natural order created 250 new nonzeros for 1,045 units of work, minimum degree 142 for 547, nested dissection 200 for 816; on larger grids nested dissection wins, with $O(N \log N)$ nonzeros and $O(N^{1.5})$ work against $O(N^2)$ and $O(N^3)$.

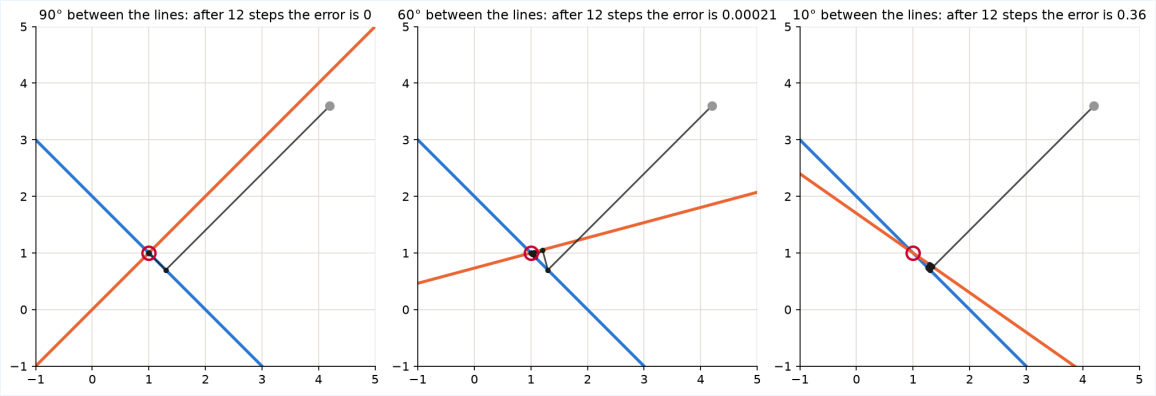


The star: eliminated centre first, the leaves become a clique and the matrix fills; centre last, nothing is created.

Section 3.

10 Solving by iterating: project onto one equation at a time

Elimination builds the answer in a fixed number of operations and, with exact arithmetic, would give it exactly. An **iterative method** starts from a guess and improves it step by step, never exactly there but as close as asked. The simplest step is a **projection**: move the guess to the nearest point of one equation's line or plane, straight down the perpendicular, so that equation now holds; then the next equation, and around again. The guess zigzags toward the crossing, and both the residual and the error fall by a fixed factor each cycle, quickly when the lines meet at a wide angle and slowly when they are nearly parallel: conditioning again. Each step costs one row, a few multiplications for a sparse matrix however many unknowns there are, and the stopping rule is the residual, the thing we can measure; at 10^{-16} there is nowhere further to go. Taking the equations at random instead of in turn works at least as well, and was analysed only recently.



Twelve projections from the same start, with the lines at 90, 60 and 10 degrees.

Section 4.

Working with the notebooks, and with an AI assistant.

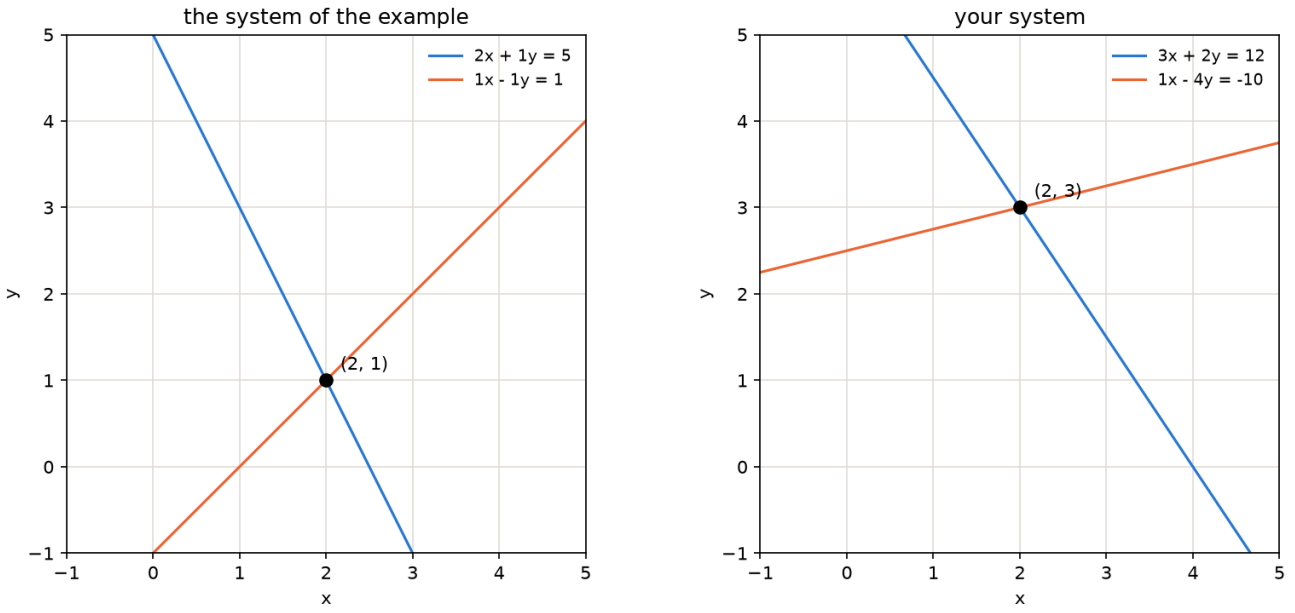
Each notebook is run from the top, one cell at a time, with Shift+Enter. A paragraph that begins with **Your turn** asks for something before you go on, and a cell under it checks what you wrote and says what to look at if something is wrong. One part, Part 4 of the second notebook, asks an AI assistant to write a function to stated requirements, and the prompt you used goes into the cell marked **Your prompt**; a precise prompt names the function, its argument, what it returns and what it may not use. The cells marked **Your thinking** ask for a few sentences in your own words; the upload is graded on the prompt and thinking cells, eight in all.

The first notebook. Linear systems

Open the first notebook (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-13-linear-systems.ipynb)

1.1 Two unknowns, two equations Part 1

An equation is **linear** when each unknown is multiplied by a number and the terms are added; one such equation with two unknowns has a whole line of solutions, and a **linear system** is several equations that must hold at once. **Elimination** finds the solution by hand: add $2x + y = 5$ and $x - y = 1$ to get $3x = 6$, so $x = 2$ and $y = 1$. The matrix form puts the numbers that multiply the unknowns in `A`, one row for each equation, the unknowns in a vector and the right-hand sides in `b`; then `A @ x` computes every left-hand side at once and the system is `A @ x == b`, which `np.linalg.solve(A, b)` solves. The part asks for the system $3x + 2y = 12$, $x - 4y = -10$ as `A2` and `b2`; the check recognizes rows and columns swapped and the signs of the second equation dropped, and the cell after it draws both systems as pairs of lines with their crossings.



Each equation is a line; each solution is a crossing.

The thinking cell asks what the rows of `A`, the vector `b` and the product `A @ x` are for a system of a thousand equations: the same thing, a thousand times over.

Code: Part 1, the system solved, the system typed, its check, and the picture — cells `solve-two`, `paste-system`, `part1-check`, `lines-picture`

```
A = np.array([[2.0, 1.0],
              [1.0, -1.0]])        # one row for each equation: the numbers that multiply x and y
```

```
b = np.array([5.0, 1.0])           # the right-hand sides
x = np.linalg.solve(A, b)
print("the solution x:", x)
print("A @ x:", A @ x, "  which is b again")
```

OUTPUT

```
the solution x: [2. 1.]
A @ x: [5. 1.]   which is b again
```

```
A2 = np.array([[3.0, 2.0],
               [1.0, -4.0]])
b2 = np.array([12.0, -10.0])
```

OUTPUT

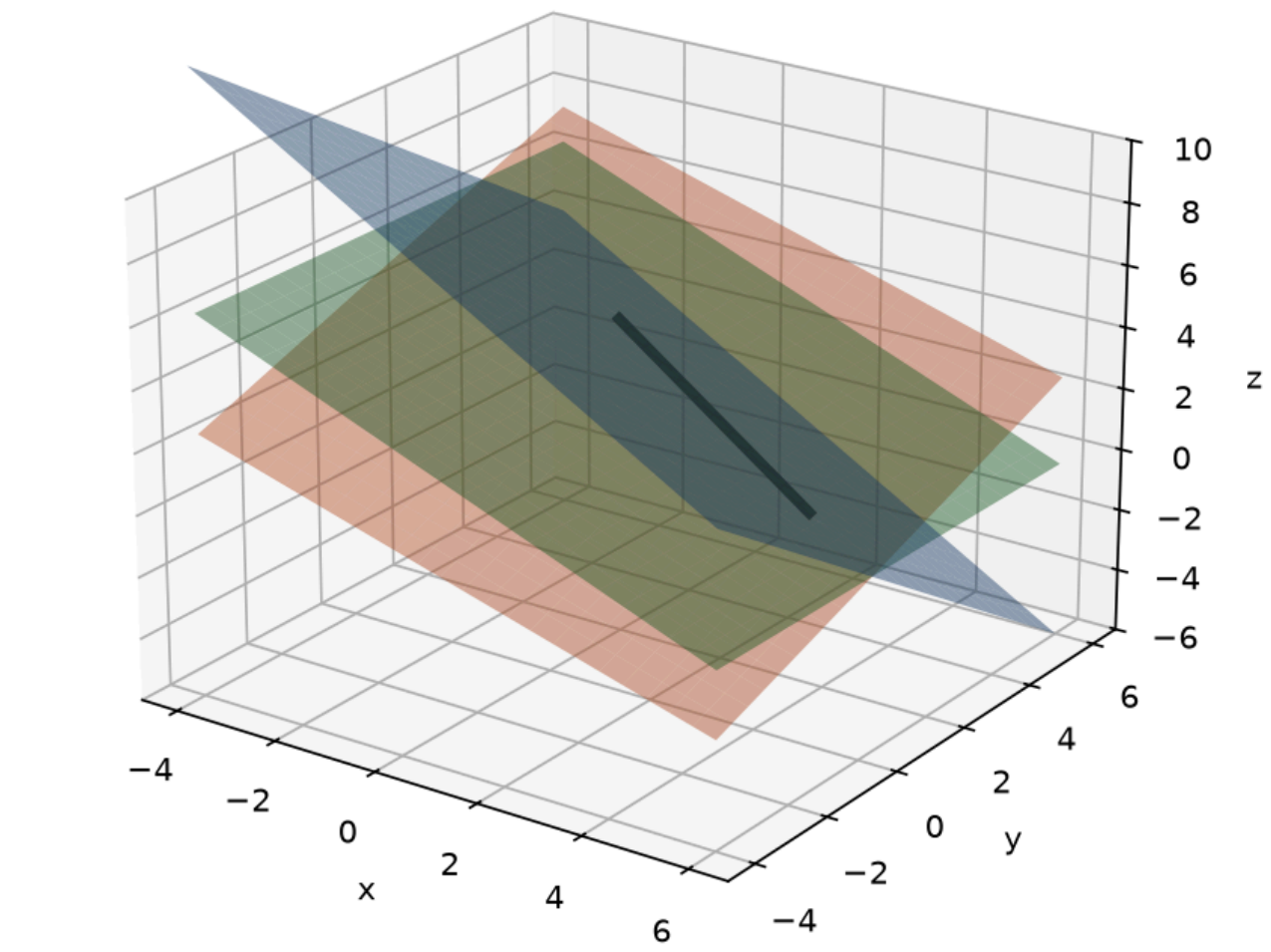
```
Correct: the solution is [2. 3.] and A2 @ x gives b2 back.
```

In the meeting.

The meeting reached this part after the walk, on the page *Linear systems, in pictures*: one equation is one line, the second equation a second line, and the solution of the system is their intersection; with three unknowns the lines become planes, and beyond that hyperplanes, as in the first unit's word, meeting at a point. "Whenever you think about a linear system, it is just lines, or planes, or hyperplanes, and it is exactly the point of intersection." The instructor added that linear systems are the subject of his own research, and that the pictures are the point of the meeting: not facts to know but images to keep.

1.2 Three unknowns, three planes Part 2

With three unknowns an equation is a plane, and the notebook draws three of them with plotly, so that the picture turns with the mouse; the solution $x = 1, y = 2, z = 3$ is the one point on all three. The first two planes meet along a line; the third cuts that line at one point. The part then asks for a third equation that is the sum of the first two, $2x + 0y + 3z = 11$, entry by entry: the third plane then contains the whole line, every point of the line is a solution, and `np.linalg.solve` stops with *Singular matrix*. The number of independent equations is the **rank**, `np.linalg.matrix_rank(A)` : 2 here, not 3. With another right-hand side, 12 for 11, the third plane is parallel to the line and misses it, and there is no solution at all; NumPy says the same thing in both cases. With two unknowns the same two things happen with two lines: the same line twice, or two parallel lines.



The third equation is the sum of the first two: the planes share a line, and the rank is 2.

Code: Part 2, the three planes, the third row, and the check that reads the error

cells `planes`, `paste-singular`, `part2-check`

```
A3 = np.array([[1.0, 1.0, 1.0],
               [1.0, -1.0, 2.0],
               [2.0, 1.0, -1.0]])
b3 = np.array([6.0, 5.0, 1.0])
x3 = np.linalg.solve(A3, b3)
print("the solution:", x3, "  A3 @ x3:", A3 @ x3)
draw_planes(A3, b3, point=x3, title="three planes, and the one point on all of them")
```

OUTPUT

```
the solution: [1. 2. 3.]   A3 @ x3: [6. 5. 1.]
```

```
third_row = [2.0, 0.0, 3.0]
third_rhs = 11.0
```

OUTPUT

```
NumPy refuses the system: Singular matrix
Correct: the three equations hold only 2 independent ones, and the planes share a line.
```

In the meeting.

On the page's third tab the three planes turned on their own, and the green one was turned toward the line of the other two: "you can change the configuration, and it changes the condition, it does not change the rank", until the green plane went through the line itself, "a very special position", where there is no single solution, the rank is 2, and the condition number is infinite; then lifted off the line, where there is no solution at all. The connection to the linear algebra course: the rank of a 2 by 2 matrix is 2, 1 or 0; it drops when rows are parallel or combine into parallel rows, and the condition number is what says how close to that a system is.

1.3 When the lines are nearly parallel Part 3

Two systems with the same solution (1, 1): in one the lines cross at a right angle; in the other, $x + y = 2$ and $x + 1.001y = 2.001$, they cross at a very small angle. The cell changes the right-hand side of the second equation by a thousandth, as a measurement error would, and solves again: the first solution moves by 0.0007, the second by 1.4142, the same change magnified about two thousand times. The **condition number**, `np.linalg.cond(A)`, is 1 for the first system and 4,002 for the second; it bounds the amplification of relative changes in b, and a condition number of 10^k can cost up to k of the sixteen digits of the solution. The part asks for a second row that makes the condition number at least a million without making the two lines the same line; the check rejects the same line and a row not close enough.

Code: Part 3, the two systems nudged, the ill-conditioned row, and its check — cells `nearly-parallel`, `paste-cond`, `part3-check`

```
A_good = np.array([[1.0, 1.0], [1.0, -1.0]])
A_bad = np.array([[1.0, 1.0], [1.0, 1.001]])
x_true = np.array([1.0, 1.0])
change = np.array([0.0, 0.001])                    # a change of one thousandth in the second right-hand
side

fig, axes = plt.subplots(1, 2, figsize=(11, 5))
for ax, A_, name in ((axes[0], A_good, "lines that cross at a right angle"), (axes[1], A_bad, "lines that
are nearly parallel")):
    b_ = A_ @ x_true
    x_moved = np.linalg.solve(A_, b_ + change)
```

```
    print(f"{name}:")
    print(f"   the solution {np.linalg.solve(A_, b_)}, after the change {x_moved}: moved by
{np.linalg.norm(x_moved - x_true):.4f}")
    print(f"   condition number {np.linalg.cond(A_):,.0f}")
    draw_lines(ax, np.vstack([A_, A_[1]]), np.append(b_, (b_ + change)[1]), name, points=[x_true,
x_moved], dashed=(2,))
fig.tight_layout()
plt.show()
```

OUTPUT

```
lines that cross at a right angle:
   the solution [1. 1.], after the change [1.0005 0.9995]: moved by 0.0007
   condition number 1
lines that are nearly parallel:
   the solution [1. 1.], after the change [0. 2.]: moved by 1.4142
   condition number 4,002
```

```
second_row = [1.0, 1.000001]
```

OUTPUT

```
Correct: the condition number is 4,000,002. A change of 0.001 in b moves the solution from [1, 1]
to [-999. 1001.].
```

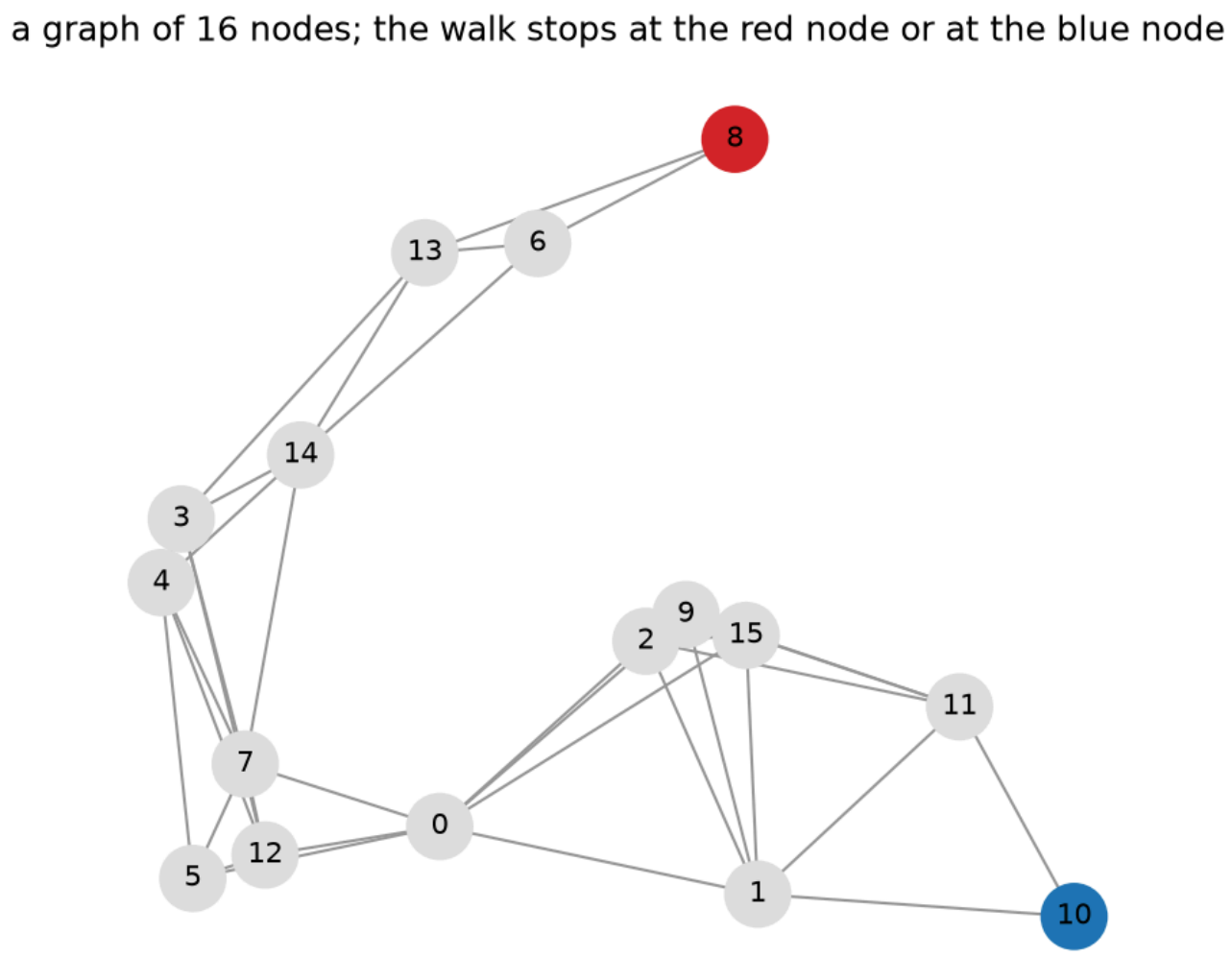
In the meeting.

The two systems side by side on the page's second tab: on the left two clearly different lines whose crossing the eye finds at once; on the right two lines that are nearly parallel, told apart only by a small difference in one coefficient, printed under the picture. The point made was general: "whenever your visual system almost tells you there is no solution, the same thing is going to happen to an algorithm": precision becomes everything, the computer loses it as the eye does, and sometimes an algorithm will report no solution where there is one. The condition number was introduced here as the measure of how parallel a system is: 1 for the right angle, "the best intersection you can hope for"; growing as the angle closes, roughly as the inverse of the cosine of the angle for two lines; infinite when the rank is not full, since the angle is then zero.

1.4 A walk on a graph Part 4

A **graph** of sixteen nodes, placed at random with an edge between any two that are close, with the two nodes farthest apart marked red and blue. From every node the notebook lets a particle walk 2,000 times, each step to a neighbour chosen at random, until it reaches red or blue, and colours the node by the fraction of the walks that reached red first; 2,000 walks leave a sampling error of a few hundredths. The question for the meeting: the sixteen probabilities also satisfy a linear system, one unknown for each node. What are its equations? The thinking cell asks for the node nearest to one

half and the node nearest to 1, and for a guess at how a node's fraction could be computed from its neighbours'.



The graph of the walk: red is node 8, blue is node 10.

Code: Part 4, the graph, the walks, and the exact solution of the deck cells `graph`, `walk`, `rc-walk-solve`

```
G = nx.random_geometric_graph(16, 0.4, seed=7)       # 16 nodes placed at random; an edge joins two nodes closer than 0.4
positions = nx.get_node_attributes(G, "pos")
lengths = dict(nx.all_pairs_shortest_path_length(G))
RED, BLUE = max(((u, v) for u in G for v in G if u < v), key=lambda pair: lengths[pair[0]][pair[1]])  # the two farthest apart
neighbours = {v: list(G.neighbors(v)) for v in G}

print(f"{G.number_of_nodes()} nodes, {G.number_of_edges()} edges. Red: node {RED}. Blue: node {BLUE}.")
print("the neighbours of each node:", {v: neighbours[v] for v in sorted(G)})
fig, ax = plt.subplots(figsize=(7, 6))
node_colours = ["#d62728" if v == RED else "#1f77b4" if v == BLUE else "#dddddd" for v in G]
nx.draw(G, positions, ax=ax, node_color=node_colours, with_labels=True, node_size=520, font_size=10,
```

```
edge_color="#999999")
ax.set_title("a graph of 16 nodes; the walk stops at the red node or at the blue node")
plt.show()
```

OUTPUT

```
16 nodes, 37 edges. Red: node 8. Blue: node 10.
the neighbours of each node: {0: [1, 2, 5, 7, 9, 12, 15], 1: [0, 2, 9, 10, 11, 15], 2: [0, 1, 9,
11, 15], 3: [4, 7, 12, 13, 14], 4: [3, 5, 7, 12, 14], 5: [0, 4, 7, 12], 6: [8, 13, 14], 7: [0, 3,
4, 5, 12, 14], 8: [6, 13], 9: [0, 1, 2, 11, 15], 10: [1, 11], 11: [1, 2, 9, 10, 15], 12: [0, 3,
4, 5, 7], 13: [3, 6, 8, 14], 14: [3, 4, 6, 7, 13], 15: [0, 1, 2, 9, 11]}
```

```
def red_first(start, walks=2000):
    """The fraction of `walks` random walks from `start` that reach the red node before the blue node."""
    reached_red = 0
    for _ in range(walks):
        v = start
        while v != RED and v != BLUE:
            v = neighbours[v][rng.integers(len(neighbours[v]))]      # a neighbour chosen at random
        reached_red += v == RED
    return reached_red / walks


fractions = {v: red_first(v) for v in sorted(G)}
print("the fraction of the walks that reached red first, from each node:")
print(", ".join(f"{v}: {fractions[v]:.2f}" for v in fractions))

fig, ax = plt.subplots(figsize=(7, 6))
nx.draw(G, positions, ax=ax, node_color=[fractions[v] for v in G], cmap="coolwarm", vmin=0, vmax=1,
        with_labels=True, node_size=520, font_size=10, edge_color="#999999")
for v in G:
    ax.annotate(f"{fractions[v]:.2f}", positions[v], xytext=(0, -18), textcoords="offset points",
ha="center", fontsize=8)
ax.set_title("the fraction of the walks that reached red first: blue is 0, red is 1")
plt.show()

print("Line for the discussion board:")
print("Red first, from each node: " + ", ".join(f"{v}: {fractions[v]:.2f}" for v in fractions)
      + f". Red is node {RED}, blue is node {BLUE}.")
```

OUTPUT

```
the fraction of the walks that reached red first, from each node:
0: 0.37, 1: 0.22, 2: 0.29, 3: 0.58, 4: 0.56, 5: 0.49, 6: 0.81, 7: 0.52, 8: 1.00, 9: 0.24, 10:
0.00, 11: 0.21, 12: 0.50, 13: 0.73, 14: 0.65, 15: 0.28
Line for the discussion board:
Red first, from each node: 0: 0.37, 1: 0.22, 2: 0.29, 3: 0.58, 4: 0.56, 5: 0.49, 6: 0.81, 7:
0.52, 8: 1.00, 9: 0.24, 10: 0.00, 11: 0.21, 12: 0.50, 13: 0.73, 14: 0.65, 15: 0.28. Red is node
8, blue is node 10.
```

```
# The answer to the question of Part 4: one unknown for each node, and one equation for each.
nodes = sorted(G)
index = {v: i for i, v in enumerate(nodes)}
M = np.zeros((len(nodes), len(nodes)))
rhs = np.zeros(len(nodes))
for v in nodes:
    i = index[v]
```

```
    M[i, i] = 1.0
    if v == RED:
        rhs[i] = 1.0                                # the walk is at red: red first, always
    elif v == BLUE:
        rhs[i] = 0.0                                # at blue: never
    else:
        for u in neighbours[v]:                     # elsewhere: the average of the neighbours
            M[i, index[u]] -= 1.0 / len(neighbours[v])
exact = np.linalg.solve(M, rhs)

example = next(v for v in nodes if v not in (RED, BLUE) and len(neighbours[v]) == 3)
print(f"node {example} has the neighbours {neighbours[example]}: its equation is "
      f"x{example} = (x{neighbours[example][0]} + x{neighbours[example][1]} + x{neighbours[example][2]}) /
3")
print(f"the row of node {example} in the matrix: 1 at column {example}, " + ", ".join(f"{M[index[example],
index[u]]:.3f} at column {u}" for u in neighbours[example]) + ", 0 elsewhere")
print(f"largest difference between the 2,000 walks and the exact solution: {max(abs(fractions[v] -
exact[index[v]]) for v in nodes):.3f}")
print()
print(f"{'node':>5s}{'neighbours':>12s}{'simulated':>11s}{'exact':>8s}")
for v in nodes:
    print(f"{v:5d}{len(neighbours[v]):12d}{fractions[v]:11.2f}{exact[index[v]]:8.3f}")

fig, ax = plt.subplots(figsize=(7, 6))
nx.draw(G, positions, ax=ax, node_color=[exact[index[v]] for v in G], cmap="coolwarm", vmin=0, vmax=1,
        with_labels=True, node_size=520, font_size=10, edge_color="#999999")
for v in G:
    ax.annotate(f"{exact[index[v]]:.3f}", positions[v], xytext=(0, -18), textcoords="offset points",
ha="center", fontsize=8)
ax.set_title("the exact probability of reaching red first, from one linear system")
plt.show()
```

OUTPUT

```
node 6 has the neighbours [8, 13, 14]: its equation is x6 = (x8 + x13 + x14) / 3
the row of node 6 in the matrix: 1 at column 6, -0.333 at column 8, -0.333 at column 13, -0.333
at column 14, 0 elsewhere
largest difference between the 2,000 walks and the exact solution: 0.032

 node  neighbours  simulated   exact
    0           7       0.37   0.363
    1           6       0.22   0.227
    2           5       0.29   0.265
    3           5       0.58   0.600
    4           5       0.56   0.554
    5           4       0.49   0.488
    6           3       0.81   0.805
    7           6       0.52   0.527
    8           2       1.00   1.000
    9           5       0.24   0.265
   10           2       0.00   0.000
   11           5       0.21   0.204
   12           5       0.50   0.507
   13           4       0.73   0.764
   14           5       0.65   0.650
   15           5       0.28   0.265
```

In the meeting.

The meeting opened here, on the page *A walk on a graph*, with the story: the graph as the web, or a graph of friendships; the red and the blue node as two pages, two parties; a reader who, unlike a real one with biases to confirm, picks a link at random each time; and the question which page the reader reaches first. "It's a modeling idea": a model of behaviour, never the behaviour itself. The same idea, a random walker over the pages of the web, was the start of PageRank and of Google, and it is the subject of an assignment of this unit. One walker was released from node 7, step by step, and ended at blue; again, and it ended at red; a hundred at once zigzagged over the graph until all but a few stragglers were absorbed, and the fraction read about 57 per cent red; from a node near the blue end, about 30; then 200 from every node, and every node took its colour. Then the exact answer was revealed, and compared: a node's simulated 24 per cent against its exact 26.

The reasoning given for the equations: from node 15, whose neighbours are 0, 1, 2, 9 and 11, the first step goes to one of the five at random, so the probability of reaching red first from 15 is the average of the five neighbours' probabilities, "you are the average of your friends, in some ways". That is one linear equation; there is one for each node, sixteen equations and sixteen unknowns, a system of linear equations, "exactly what you hear about in linear algebra", and solving it gives the exact numbers without a trillion simulations: 36.3 per cent from node 0, 22.7 from node 1. Why it matters: when the numbers decide how a budget is spread over neighbourhoods, the difference between an estimate and the exact value can change the decision.

The second notebook. Solvers and sparse matrices

Open the second notebook (https://colab.research.google.com/github/ikoutis/cs301-labs/blob/main/unit-2/notebooks/meeting-13-solvers.ipynb)

2.1 Elimination, with loops Part 1

The method of Part 1 of the first notebook, written for n equations, is **Gaussian elimination**. **Forward elimination** goes through the unknowns in order: for unknown k it subtracts a multiple of equation k, the **pivot row**, from every equation below it, so that the unknown disappears from them; the number it divides by, `A[k, k]`, is the **pivot**. When it is done the matrix is **upper triangular**, and **back substitution** solves the last equation for the last unknown and climbs. The cell writes forward elimination with three loops, like the layer of neurons of meeting 12, so that its operations can be counted: the innermost line runs for every unknown, every row below it and every column from it on, about $n^3 / 3$ times, two arithmetic operations each, $\mathbf{O(n^3)}$. The part asks for the line of back substitution, `x[i] = (c[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]`, and the check

names the three usual mistakes: no division by the pivot, the unknowns before i in place of those after, and a plus for the minus. The loops are then timed at $n = 50$, 100 and 200: on this machine 0.537 seconds at 200, and the class's Colab sessions took 1.4; a system of 4,000 unknowns would take the loops hours.

Code: Part 1, elimination with three loops, the line of back substitution, its check, and the loops timed

cells `elimination`, `paste-back`, `part1-check`, `time-loops`

```
def eliminate(A, b):
    """Forward elimination: a copy of A made upper triangular, and b changed with it, so that the
solutions stay the same."""
    A, b = A.astype(float).copy(), b.astype(float).copy()
    n = len(b)
    for k in range(n):                      # the unknown to remove from the rows below row k
        for i in range(k + 1, n):           # every row below row k
            factor = A[i, k] / A[k, k]      # the multiple of row k that removes the unknown from row i
            for j in range(k, n):           # every column from k on; the columns before k are already 0
                A[i, j] -= factor * A[k, j]
            b[i] -= factor * b[k]
    return A, b


A, b = random_system(4)
U, c = eliminate(A, b)
np.set_printoptions(precision=3, suppress=True)
print("the matrix after elimination, upper triangular:")
print(U)
print("the right-hand side, changed with it:", c)
```

OUTPUT

```
the matrix after elimination, upper triangular:
[[ 4.637  0.27   0.041  0.017]
 [ 0.     4.865  0.599  0.727]
 [ 0.     0.     4.7   -0.134]
 [ 0.     0.     0.     4.196]]
the right-hand side, changed with it: [0.863 0.39  0.126 0.245]
```

```
def back_substitute(U, c):
    """The solution of U x = c for an upper triangular U, from the last unknown to the first."""
    n = len(c)
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (c[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x
```

OUTPUT

```
Correct: back_substitute finds the solution; eliminate and back_substitute together solve the
system.
```

OUTPUT

n	passes, about n^3 / 3	seconds	ratio to the first	nanoseconds per pass
50	41,666	0.009	1.0	209
100	333,333	0.071	8.1	213
200	2,666,666	0.537	61.8	201

In the meeting.

Elimination was shown on the page's fourth tab as a picture before it was counted: the augmented matrix of the three-plane system beside the planes, and step 1, the first row subtracted from the second to make a zero in the first column. "When I'm doing that, I'm really subtracting one plane from the other": one plane moved, pivoting about the line it shares with the other, and the solution did not move. Step 2 moved the green plane for the second zero, step 3 subtracted the second equation for the third: "Gaussian elimination is this idea that I'm taking the matrix, which initially is dense, and create these zeros that are going to make the solution faster later, and easier." Then the count, on the whiteboard, from the class: the first row subtracted from the others is $n-1$ subtractions of vectors, n numbers each, about n^2; the next step works on the submatrix of size $n-1$, about $(n-1)^2$; then $(n-2)^2$, down to 1^2; the sum of the squares is about n^3, a student's bound before the exact sum, and that is why elimination is three loops.

2.2 Three ways to solve Part 2

The same system solved three ways: the loops of Part 1; the **inverse**, `np.linalg.inv(A) @ b`, which is elimination applied to n right-hand sides at once, about $2n^3$ operations, three times one elimination, and then a product; and `np.linalg.solve(A, b)`, elimination in compiled code with rows swapped where the pivot is small. The cell times the last two at n from 500 to 4,000. On this machine, with 32 logical cores, the inverse took 0.850 seconds at 4,000 and `solve` 0.430; on Colab's two cores, in the class's posts, about 6 and 1.5. The part asks for the ratio of the two at 4,000 and for the growth of `solve` from 2,000 to 4,000, read from the table; the growth is about 8, the count's. For $Ax = b$, prefer `solve`: less work, and more accurate, as the next part shows; and for a sparse matrix the inverse is usually full. The thinking cell asks where the difference between two O(n³) methods comes from: the constant in front, which O( ) drops.

Code: Part 2, the timing, the two numbers, and their check

cells `time-three`, `answers-three`, `part2-check`

```
sizes = [500, 1000, 2000, 4000]
seconds = {"inverse": [], "solve": []}
for n in sizes:
    A, b = random_system(n)
    seconds["inverse"].append(best_time(lambda: np.linalg.inv(A) @ b, repeats=2))
```

```
    seconds["solve"].append(best_time(lambda: np.linalg.solve(A, b), repeats=2))

print(f"{'n':>6s}{'np.linalg.inv(A) @ b':>22s}{'np.linalg.solve(A, b)':>24s}{'inverse / solve':>18s}")
for i, n in enumerate(sizes):
    print(f"{n:6d}{seconds['inverse'][i]:22.4f}{seconds['solve'][i]:24.4f}{seconds['inverse'][i] /
seconds['solve'][i]:18.1f}")

fig, ax = plt.subplots(figsize=(8, 4.6))
ax.plot(loop_sizes, loop_times, marker="^", color="#888888", label="Python: three loops")
ax.plot(sizes, seconds["inverse"], marker="o", color="#eb6834", label="np.linalg.inv(A) @ b")
ax.plot(sizes, seconds["solve"], marker="s", color="#2a78d6", label="np.linalg.solve(A, b)")
ax.set_xscale("log", base=2)
ax.set_yscale("log")
ax.set_xticks(loop_sizes + sizes)
ax.set_xticklabels([str(n) for n in loop_sizes + sizes])
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda value, position: f"{value:g}"))
ax.set_xlabel("n: the number of equations and of unknowns")
ax.set_ylabel("seconds for one solution")
ax.set_title("Three ways to solve A x = b (each step up the axis is 10 times longer)")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
plt.show()

print("Line for the discussion board:")
print(f"Solving n = 4,000 equations: the inverse {seconds['inverse'][-1]:.3f} s, solve {seconds['solve']
[-1]:.3f} s; "
      f"the loops of Python at n = 200: {loop_times[-1]:.2f} s, so about {duration(loop_times[-1] * 8000)}
at n = 4,000.")
```

OUTPUT

```
     n  np.linalg.inv(A) @ b   np.linalg.solve(A, b)   inverse / solve
   500                 0.1076                  0.0812               1.3
  1000                 0.1500                  0.1320               1.1
  2000                 0.3019                  0.2244               1.3
  4000                 0.8505                  0.4305               2.0
Line for the discussion board:
Solving n = 4,000 equations: the inverse 0.850 s, solve 0.430 s; the loops of Python at n = 200:
0.54 s, so about 71.6 minutes at n = 4,000.
```

```
INV_OVER_SOLVE = seconds["inverse"][-1] / seconds["solve"][-1]
SOLVE_GROWTH = seconds["solve"][-1] / seconds["solve"][-2]
```

OUTPUT

```
Correct: the inverse takes 2.0 times as long as solve at n = 4,000, and from 2,000 to 4,000 the
time of solve grows by a factor of 1.9.
```

In the meeting.

The race on the first tab of *Solvers, round-off and sparse matrices, in motion*, first at n = 2,000, then at 4,000: the loops of Python barely moved, "excruciatingly slow, and we try to avoid it"; the inverse, the formula $x = A^{-1}b$ that linear algebra gives, finished well after `solve`; and elimination "in a very well-organized way", `np.linalg.solve`, won. "When you want to solve a system like that, you just go for doing elimination, and you are careful about using it in NumPy." Pivots came up from the class, as the row with the largest entry, chosen for stability; the meeting came back to the word later with a different meaning, the order of elimination.

2.3 Round-off, and the condition number Part 3

A float64 number keeps about sixteen decimal digits, and every operation rounds to that. The cell builds systems whose solution it knows, `b = A @ x_true`, for a matrix of 100 nearly identical equations, `np.ones((n, n)) + eps * np.eye(n)`, the nearly parallel lines of the first notebook in a hundred dimensions, with the condition number about $100 / \varepsilon$; it solves each with `solve` and with the inverse and prints the condition number, the **relative error** of each solution and its **residual**, the length of $Ax - b$ over the length of b. The rule of thumb, digits of the solution $\approx 16 - \log_{10}$(condition number), holds up: about 12 digits at 10^4, 8 at 10^8, 4 at 10^{12}, none at 10^{16}; the inverse is less accurate throughout; and the residual of `solve` stays near 10^{-16} even when its solution is wrong in every digit. The part asks how many digits can be trusted at a condition number of 10^{10}: about 6. The thinking cell asks how a candidate can satisfy the equations almost exactly and still be far from the true solution: on nearly parallel lines, every point near both lines has a small residual, and the band of such points is long.

Code: Part 3, the round-off table, the digits left, and the check — cells `roundoff`, `answer-digits`, `part3-check`

```
n = 100
x_true = rng.random(n)
error = lambda x: np.linalg.norm(x - x_true) / np.linalg.norm(x_true)
print(f"{'eps':>8s}{'condition number':>18s}{'error of solve':>16s}{'error of inverse':>18s}{'residual of
solve':>19s}{'residual of inverse':>21s}")
for eps in [1e-2, 1e-6, 1e-10, 1e-14]:
    A = np.ones((n, n)) + eps * np.eye(n)
    b = A @ x_true
    x_solve = np.linalg.solve(A, b)
    x_inverse = np.linalg.inv(A) @ b
    residual = lambda x: np.linalg.norm(A @ x - b) / np.linalg.norm(b)
    print(f"{eps:8.0e}{np.linalg.cond(A):18.1e}{error(x_solve):16.1e}{error(x_inverse):18.1e}"
          f"{residual(x_solve):19.1e}{residual(x_inverse):21.1e}")
```

OUTPUT

```
     eps  condition number  error of solve  error of inverse  residual of solve  residual of
inverse
   1e-02           1.0e+04         4.7e-13           7.1e-12            1.1e-16
3.2e-13
   1e-06           1.0e+08         6.7e-09           9.9e-08            2.1e-16
1.0e-08
   1e-10           1.0e+12         5.3e-05           4.1e-03            2.0e-16
1.6e-04
   1e-14           1.0e+16         6.3e-01           1.9e+01            3.9e-16
1.0e+00
```

```
DIGITS_LEFT = 6
```

OUTPUT

```
Correct: about 6 digits, 16 minus 10.
```

In the meeting.

The question put to the class: having computed a solution, can you compute its error? Not exactly, a student said, "because then you'd know the exact solution anyway", which is the whole point: knowing the error would hand us the solution we were looking for, so the error is never known. What can be measured is a proxy: with the system $Ax = b$ and a computed x', the quantity $b - Ax'$, the residual, "and that's going to tell me how good my solution is". On the page's second tab the candidate in the red band had a residual printed beside it while the true solution stayed hidden; revealed, the error was many times the residual. The warning: the residual can be small while the error is high, the proxy does not always track the error, and the error is tied to the condition number: a matrix that is nearly rank-deficient loses digits. A promise for later in the unit: how a matrix can be stored with less precision and still serve machine learning well.

2.4 Matrices that are mostly zeros Part 4

Many systems involve only a few unknowns in each equation, and their matrices are mostly zero: **sparse**. The example is a **grid** of $n \times n$ points, each tied to its four neighbours: $N = n^2$ unknowns, each equation with 4 on the diagonal and -1 for each neighbour, at most five nonzeros a row however large the grid; equations of this shape describe heat on a plate, pressure in a flow, and the walk of the first notebook. The dense matrix of a 100×100 grid holds 100 million numbers, 800 megabytes, of which about 50,000 are nonzero. The cell builds the 3×3 grid's matrix in COO and converts it to CSR and CSC, printing the arrays of each: **COO**, three arrays with the row, the column and the value of each nonzero, convenient for building; **CSR**, the values and their columns row by row, and `indptr`, $N + 1$ numbers that say where each row begins, suited to products; **CSC**,

the same by columns, used by direct solvers. Dense storage costs 8 bytes a number; CSR 12 bytes a nonzero plus 4 for each row pointer. The part asks an AI assistant for `dense_to_csr`, which builds the three arrays without scipy; the check compares them with scipy's and reads the function's syntax for a call to scipy. The memory table ends at the 3,000 × 3,000 grid: for a million unknowns, 8 TB dense against 64 MB in CSR.

Code: Part 4, the grid's matrix in three formats, the function written to the requirements, its check, and the memory table

cells `formats`, `paste-csr`, `part4-check`, `memory`

```
def grid_matrix(n):
    """The matrix of an n x n grid in COO format: 4 on the diagonal, -1 for each of the four neighbours of a point."""
    rows, cols, values = [], [], []
    for i in range(n):
        for j in range(n):
            k = i * n + j                           # the point in row i and column j of the grid is unknown k
            rows.append(k)
            cols.append(k)
            values.append(4.0)
            for di, dj in ((-1, 0), (1, 0), (0, -1), (0, 1)):
                if 0 <= i + di < n and 0 <= j + dj < n:
                    rows.append(k)
                    cols.append((i + di) * n + j + dj)
                    values.append(-1.0)
    return sparse.coo_matrix((values, (rows, cols)), shape=(n * n, n * n))


A_coo = grid_matrix(3)
A_csr = A_coo.tocsr()
A_csc = A_coo.tocsc()
np.set_printoptions(linewidth=140)
print("the dense matrix of the 3 x 3 grid, 9 rows and 9 columns:")
print(A_coo.toarray().astype(int))
print(f"nonzeros: {A_coo.nnz} of {A_coo.shape[0] * A_coo.shape[1]} numbers")
print("COO  row    ", A_coo.row)
print("     col    ", A_coo.col)
print("     data   ", A_coo.data.astype(int))
print("CSR  indptr ", A_csr.indptr, "  where each row begins in the two arrays below; the last entry is the number of nonzeros")
print("     indices", A_csr.indices, "  the column of each nonzero, row by row")
print("     data   ", A_csr.data.astype(int))
print("CSC  indptr ", A_csc.indptr, "  where each column begins")
print("     indices", A_csc.indices, "  the row of each nonzero, column by column")
print("     data   ", A_csc.data.astype(int))
```

OUTPUT

```
the dense matrix of the 3 x 3 grid, 9 rows and 9 columns:
[[ 4 -1  0 -1  0  0  0  0  0]
 [-1  4 -1  0 -1  0  0  0  0]
 [ 0 -1  4  0  0 -1  0  0  0]
 [-1  0  0  4 -1  0 -1  0  0]
 [ 0 -1  0 -1  4 -1  0 -1  0]
 [ 0  0 -1  0 -1  4  0  0 -1]
 [ 0  0  0 -1  0  0  4 -1  0]
 [ 0  0  0  0 -1  0 -1  4 -1]
 [ 0  0  0  0  0 -1  0 -1  4]]
nonzeros: 33 of 81 numbers
COO  row     [0 0 0 1 1 1 1 2 2 2 3 3 3 3 4 4 4 4 4 5 5 5 5 6 6 6 7 7 7 7 8 8 8]
     col     [0 3 1 1 4 0 2 2 5 1 3 0 6 4 4 1 7 3 5 5 2 8 4 6 3 7 7 4 6 8 8 5 7]
     data    [ 4 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 -1 -1  4 -1 -1 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1
-1 -1  4 -1 -1]
CSR  indptr  [ 0  3  7 10 14 19 23 26 30 33]   where each row begins in the two arrays below; the
last entry is the number of nonzeros
     indices [0 1 3 0 1 2 4 1 2 5 0 3 4 6 1 3 4 5 7 2 4 5 8 3 6 7 4 6 7 8 5 7 8]   the column of
each nonzero, row by row
     data    [ 4 -1 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 -1 -1 -1  4 -1 -1 -1 -1  4 -1 -1  4 -1 -1 -1
4 -1 -1 -1  4]
CSC  indptr  [ 0  3  7 10 14 19 23 26 30 33]   where each column begins
     indices [0 1 3 0 1 2 4 1 2 5 0 3 4 6 1 3 4 5 7 2 4 5 8 3 6 7 4 6 7 8 5 7 8]   the row of
each nonzero, column by column
     data    [ 4 -1 -1 -1  4 -1 -1 -1  4 -1 -1  4 -1 -1 -1 -1  4 -1 -1 -1 -1  4 -1 -1  4 -1 -1 -1
4 -1 -1 -1  4]
```

```python
def dense_to_csr(A):
    """The three arrays of the CSR format of a dense array: data, indices, indptr."""
    data, indices, indptr = [], [], [0]
    for row in A:
        nonzero = np.nonzero(row)[0]
        data.extend(row[nonzero])
        indices.extend(nonzero)
        indptr.append(len(data))
    return np.array(data, dtype=float), np.array(indices, dtype=int), np.array(indptr, dtype=int)
```

OUTPUT

```
Correct: dense_to_csr gives the same three arrays as scipy.sparse.csr_matrix.
```

OUTPUT

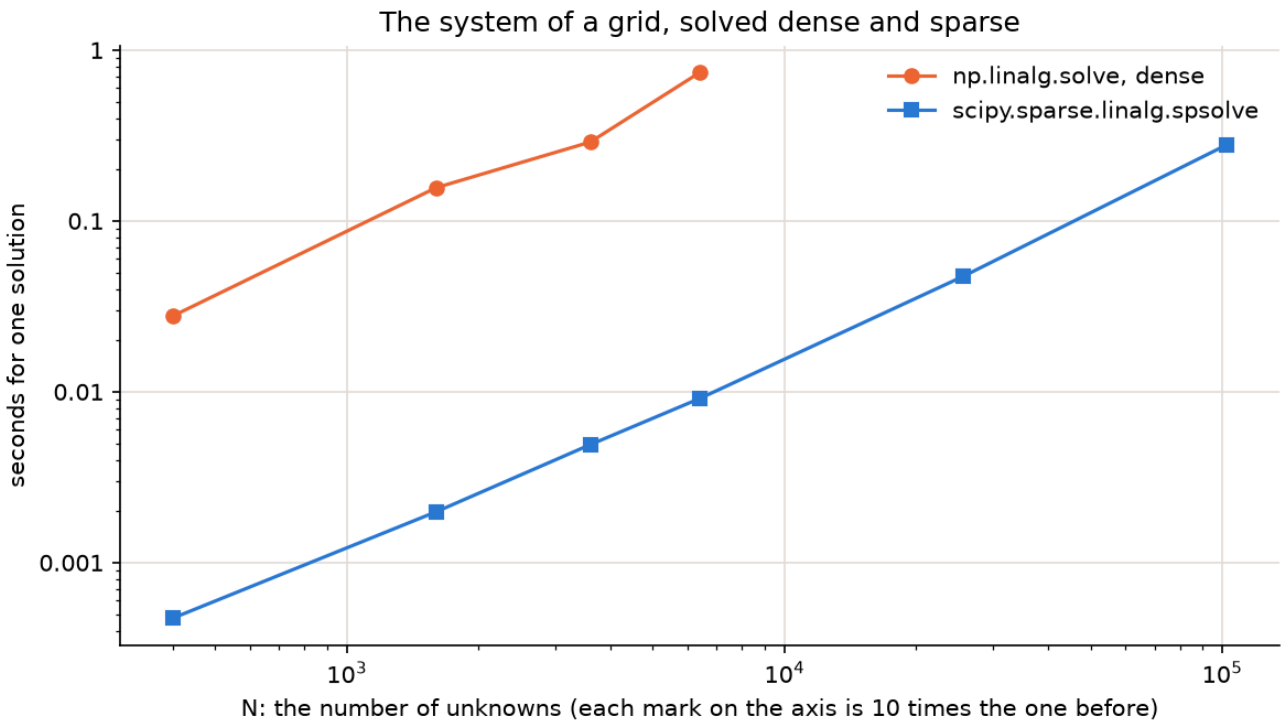
```
    grid  N = unknowns    nonzeros   dense, 8 bytes a number   CSR, 12 bytes a nonzero
   10 x 10            100         460                    80 KB                  5.92 KB
   30 x 30            900       4,380                  6.48 MB                  56.2 KB
  100 x 100        10,000      49,600                   800 MB                   635 KB
  300 x 300        90,000     448,800                  64.8 GB                  5.75 MB
 1000 x 1000    1,000,000   4,996,000                     8 TB                    64 MB
 3000 x 3000    9,000,000  44,988,000                   648 TB                   576 MB
check on the 100 x 100 grid, built: 49600 nonzeros
```

In the meeting.

The 3 × 3 grid's matrix on the page's third tab, its blank cells read by the class as zeros, and the question of how many zeros the web graph has. The count for a social network: N accounts could be joined in N^2 ways, a billion squared for a billion accounts, which no memory could hold; each follows a few hundred, so the whole graph fits in a modest data centre, and copies of it sit in several places for safety. The formats were shown, not dwelt on: "we are not storing the zeros", only the values, with a little information about where each row or column starts. Then the two products animated: dense, every entry of the matrix read and multiplied, "zero times something is equal to zero", so the zeros are neutral, algebraically they do nothing, and the only operation that matters in linear algebra, machine learning and AI is a matrix times a vector; CSR, the cursor landing only on the nonzeros and finishing far sooner. "This storage saves space, saves a lot of memory, it also saves a lot of computation." A note from research: how to store a sparse matrix so that the product runs fast is still worked on, the instructor's own paper on a storage format appeared at the supercomputing conference around 2010, and a random matrix, unlike a grid's, has no pattern of access that the cache can follow, meeting 12's lesson again.

2.5 Solving, dense and sparse Part 5

`scipy.sparse.linalg.spsolve` eliminates while skipping the zeros; `np.linalg.solve` needs the dense array. The cell times both on grids from $n = 20$ to 80, $N = 400$ to 6,400, and the sparse solver alone up to $n = 320$, $N = 102{,}400$, where the dense matrix would need 84 gigabytes. On this machine the 80 × 80 grid took 0.739 seconds dense and 0.0091 sparse; in the class's posts, about 6 seconds and a few hundredths. Dense elimination is $O(N^3)$, so quadrupling the unknowns multiplies the work by 64; sparse elimination grows far more slowly on the grid, but it is not O(nnz) in general: elimination creates new nonzeros, **fill-in**, whose number depends on the order, the subject of section 3.



The grid's system solved dense and sparse, on the machine that built these notes.

Code: Part 5, the grids solved both ways — cell `time-solvers`

```
grid_sizes = [20, 40, 60, 80, 160, 320]
dense_sizes = [20, 40, 60, 80]                    # the dense matrix of the 80 x 80 grid is 328 MB; the
next would be 5 GB
unknowns, dense_seconds_solve, sparse_seconds_solve = [], [], []
for n in grid_sizes:
    A_sparse = grid_matrix(n).tocsc()
    N = n * n
    b = rng.random(N)
    unknowns.append(N)
    sparse_seconds_solve.append(best_time(lambda: sparse_linalg.spsolve(A_sparse, b), repeats=2))
    if n in dense_sizes:
        A_dense = A_sparse.toarray()
        dense_seconds_solve.append(best_time(lambda: np.linalg.solve(A_dense, b), repeats=2))
        del A_dense

print(f"{'grid':>10s}{'N':>9s}{'nonzeros':>10s}{'np.linalg.solve, dense':>25s}{'spsolve, sparse':>18s}")
for i, n in enumerate(grid_sizes):
    dense_text = f"{dense_seconds_solve[i]:.4f} s" if i < len(dense_seconds_solve) else "too large to
store"
    print(f"{n:5d} x {n:<3d}{unknowns[i]:9,d}{grid_matrix(n).nnz:10,d}{dense_text:>25s}
{sparse_seconds_solve[i]:16.4f} s")

fig, ax = plt.subplots(figsize=(8, 4.6))
ax.plot(unknowns[:len(dense_seconds_solve)], dense_seconds_solve, marker="o", color="#eb6834",
label="np.linalg.solve, dense")
ax.plot(unknowns, sparse_seconds_solve, marker="s", color="#2a78d6", label="scipy.sparse.linalg.spsolve")
ax.set_xscale("log")
ax.set_yscale("log")
```

```
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda value, position: f"{value:g}"))
ax.set_xlabel("N: the number of unknowns (each mark on the axis is 10 times the one before)")
ax.set_ylabel("seconds for one solution")
ax.set_title("The system of a grid, solved dense and sparse")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
plt.show()

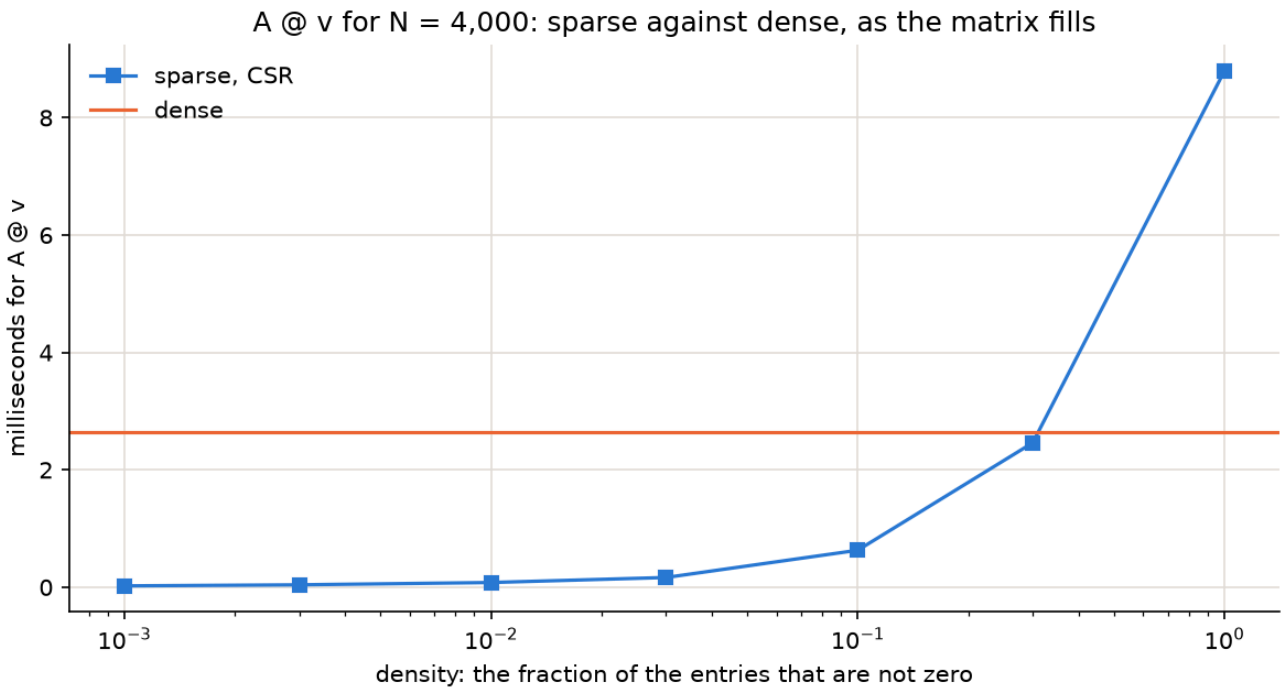
print("Line for the discussion board:")
print(f"The grid of 80 x 80, N = 6,400: dense {dense_seconds_solve[-1]:.3f} s, sparse
{sparse_seconds_solve[3]:.4f} s; "
      f"the grid of 320 x 320, N = 102,400: sparse {sparse_seconds_solve[-1]:.3f} s, dense not possible.")
```

OUTPUT

```
      grid        N  nonzeros   np.linalg.solve, dense   spsolve, sparse
   20 x 20       400     1,920                 0.0277 s          0.0005 s
   40 x 40     1,600     7,840                 0.1565 s          0.0020 s
   60 x 60     3,600    17,760                 0.2897 s          0.0049 s
   80 x 80     6,400    31,680                 0.7390 s          0.0091 s
  160 x 160   25,600   127,360       too large to store          0.0474 s
  320 x 320  102,400   510,720       too large to store          0.2791 s
Line for the discussion board:
The grid of 80 x 80, N = 6,400: dense 0.739 s, sparse 0.0091 s; the grid of 320 x 320, N =
102,400: sparse 0.279 s, dense not possible.
```

2.6 The cost of a product, as the matrix fills Part 6

A product `A @ v` costs N^2 multiply-adds for a dense matrix, zeros included, and one for each stored nonzero in CSR, $\mathbf{O(nnz)}$. The cell times both on random matrices of size 4,000 at densities from one in a thousand to all, and asks for the smallest density at which the dense product is faster, `CROSSOVER_DENSITY`, or `None` if the sparse product won throughout. Each stored nonzero costs more than a dense entry, since its column is read with it and the entries of v are fetched in no order, so the crossover is at some tens of per cent on most machines, and on Colab's two cores several groups found the sparse product ahead at every density of the table. The grid is very sparse, about five nonzeros in N. The last thinking cell asks when a matrix is worth storing sparse: when the nonzeros, which set both the bytes and the operations, are few.



The product as the matrix fills, on the machine that built these notes.

Code: Part 6, the products timed, the crossover, and its check

cells `time-products`, `answer-crossover`, `part6-check`

```
N = 4000
v = rng.random(N)
A_dense = rng.random((N, N))
dense_seconds = best_time(lambda: A_dense @ v, repeats=5)
densities = [0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0]
sparse_seconds, nonzeros = [], []
for density in densities:
    A_sparse = sparse.random(N, N, density=density, format="csr", random_state=1)
    nonzeros.append(A_sparse.nnz)
    sparse_seconds.append(best_time(lambda: A_sparse @ v, repeats=5))
del A_sparse

print(f"dense product, {N:,} x {N:,} numbers: {dense_seconds * 1e3:.2f} milliseconds, {dense_seconds / N
** 2 * 1e9:.2f} nanoseconds a number")
print(f"{'density':>9s}{'nonzeros':>12s}{'sparse product, ms':>21s}{'ns a nonzero':>14s}{'faster':>9s}")
for density, nnz, s in zip(densities, nonzeros, sparse_seconds):
    print(f"{density:9g}{nnz:12,d}{s * 1e3:21.2f}{s / nnz * 1e9:14.2f}{'sparse' if s < dense_seconds else
'dense':>9s}")

fig, ax = plt.subplots(figsize=(8, 4.4))
ax.plot(densities, [s * 1e3 for s in sparse_seconds], marker="s", color="#2a78d6", label="sparse, CSR")
ax.axhline(dense_seconds * 1e3, color="#eb6834", label="dense")
ax.set_xscale("log")
ax.set_xlabel("density: the fraction of the entries that are not zero")
ax.set_ylabel("milliseconds for A @ v")
ax.set_title(f"A @ v for N = {N:,}: sparse against dense, as the matrix fills")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
```

```
ax.legend(frameon=False)
fig.tight_layout()
plt.show()
```

OUTPUT

```
dense product, 4,000 x 4,000 numbers: 2.64 milliseconds, 0.17 nanoseconds a number
  density    nonzeros   sparse product, ms  ns a nonzero   faster
    0.001      16,000                0.02          1.46   sparse
    0.003      48,000                0.04          0.85   sparse
     0.01     160,000                0.08          0.50   sparse
     0.03     480,000                0.17          0.35   sparse
      0.1   1,600,000                0.63          0.39   sparse
      0.3   4,800,000                2.46          0.51   sparse
        1  16,000,000                8.80          0.55    dense
```

```
CROSSOVER_DENSITY = next((d for d, s in zip(densities, sparse_seconds) if dense_seconds < s), None)
```

OUTPUT

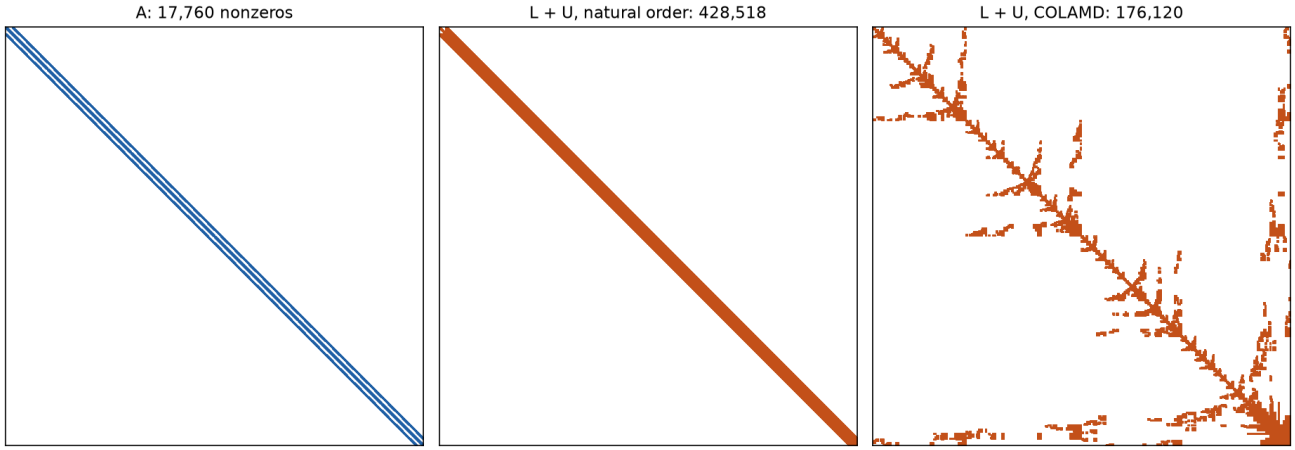
```
Correct: from a density of 1 on, the dense product is faster on this machine.
```

The two posts.

The first notebook asks for one post on the board of the meeting, after Part 4: the three planes with the solution point, the two pairs of lines with their condition numbers, and the coloured graph with the line printed under it and the group's guess at the equations. The second asks for a reply after Part 6: the chart of the three ways to solve with its line, the chart of the grid solved dense and sparse with its line, and the crossover density. A post earns the attention mark of the meeting for everyone it names; the posts of this meeting carried the Colab timings quoted above.

Fill-in: the order of elimination

A matrix is a graph and a graph is a matrix: an off-diagonal nonzero $A[i,j]$ is an edge between unknowns i and j. Eliminating unknown k subtracts a multiple of its row from every neighbour's row, and wherever row k has a nonzero, each neighbour's row gets one: afterwards every pair of neighbours of k is joined. The new nonzeros are the **fill-in**, and sparse elimination must store them and work on them. The star makes the point exactly: a centre joined to five leaves has 16 nonzeros; eliminate the centre first and the five leaves become a clique, 36 nonzeros, the whole matrix; eliminate the centre last, after the leaves, and the count stays at 16. On a star of a million leaves the first order runs out of memory on any machine, and the second takes milliseconds. The order of elimination, the choice of the *pivot* in this second sense of the word, decides the cost.



The 60 × 60 grid's matrix, and its factors after elimination in the natural order and in SuperLU's default order.

On the 60 × 60 grid, 3,600 unknowns, scipy's `splu` gives the factors in three orders: in the natural order, row by row of the grid, 24.1 times the nonzeros of A; in COLAMD, the fill-reducing order that SuperLU and `spsolve` use by default, 9.9 times; in a minimum-degree order 5.9 times. Fewer nonzeros in the factors means less to store and less to do, and the solve time follows them.

Code: the star in two orders, the grid's factors in three, and the solve times that follow the fill

cells `dl-star`, `dl-grid-fill`, `dl-grid-time`

```
import networkx as nx


def fill_pattern(A_):
    """Where elimination without pivoting leaves nonzeros: the pattern of the upper triangle after
eliminate, plus the
    multipliers below it (the L part), as a 0/1 array."""
    n_ = A_.shape[0]
    B = A_.astype(float).copy()
    pattern = (B != 0)
    for k in range(n_):
        for i in range(k + 1, n_):
            if B[i, k] != 0:
                factor = B[i, k] / B[k, k]
                B[i, k:] -= factor * B[k, k:]
                pattern[i, k:] |= (B[k, k:] != 0)       # the row gains every nonzero of row k from k on
    return pattern


def star_matrix(order):
    """The matrix of a star with 6 nodes (centre and five leaves), rows and columns in the given order of
the nodes."""
    S = np.eye(6) * 6.0
    for leaf in range(1, 6):
        S[0, leaf] = S[leaf, 0] = -1.0
    return S[np.ix_(order, order)]


first = fill_pattern(star_matrix([0, 1, 2, 3, 4, 5]))      # the centre eliminated first
```

```
last = fill_pattern(star_matrix([1, 2, 3, 4, 5, 0]))       # the centre eliminated last
print("nonzeros of the star matrix: 16;  after elimination, centre first:", int(first.sum()), " centre
last:", int(last.sum()))

G_star = nx.star_graph(5)
pos = nx.spring_layout(G_star, seed=2)
fig, axes = plt.subplots(1, 4, figsize=(12.5, 3.3))
nx.draw(G_star, pos, ax=axes[0], node_color=["#eb6834"] + ["#cfe1f2"] * 5, with_labels=True,
node_size=380, edge_color="#999999")
axes[0].set_title("the star: a matrix of 6 rows", fontsize=11)
G_clique = nx.complete_graph(range(1, 6))
nx.draw(G_clique, pos, ax=axes[1], node_color="#cfe1f2", with_labels=True, node_size=380,
edge_color="#c6531a")
axes[1].set_title("the centre eliminated first:\nits neighbours all joined", fontsize=11)
axes[2].spy(first, markersize=9, color="#c6531a")
axes[2].set_title(f"centre first: {int(first.sum())} nonzeros", fontsize=11)
axes[3].spy(last, markersize=9, color="#1f5fa8")
axes[3].set_title(f"centre last: {int(last.sum())} nonzeros", fontsize=11)
for ax in axes[2:]:
    ax.set_xticks([])
    ax.set_yticks([])
fig.tight_layout()
plt.show()
```

OUTPUT

```
nonzeros of the star matrix: 16;  after elimination, centre first: 36  centre last: 16
```

```
A60 = grid_matrix(60).tocsc()
factors = {}
for name, spec in (("natural order", "NATURAL"), ("COLAMD (SuperLU's default)", "COLAMD"), ("minimum
degree", "MMD_AT_PLUS_A")):
    lu = sparse_linalg.splu(A60, permc_spec=spec)
    factors[name] = (lu.L + lu.U).tocsr()
print(f"the 60 x 60 grid: N = {A60.shape[0]:,}, {A60.nnz:,} nonzeros in A")
print(f"{'order of elimination':30s}{'nonzeros of L + U':>20s}{'times those of A':>18s}")
for name, F in factors.items():
    print(f"{name:30s}{F.nnz:20,d}{F.nnz / A60.nnz:18.1f}")

fig, axes = plt.subplots(1, 3, figsize=(12.5, 4.3))
axes[0].spy(A60, markersize=0.3, color="#1f5fa8")
axes[0].set_title(f"A: {A60.nnz:,} nonzeros", fontsize=11)
for ax, name in zip(axes[1:], ("natural order", "COLAMD (SuperLU's default)")):
    ax.spy(factors[name], markersize=0.3, color="#c6531a")
    ax.set_title(f"L + U, {name.split(' (')[0]}: {factors[name].nnz:,}", fontsize=11)
for ax in axes:
    ax.set_xticks([])
    ax.set_yticks([])
fig.tight_layout()
plt.show()
```

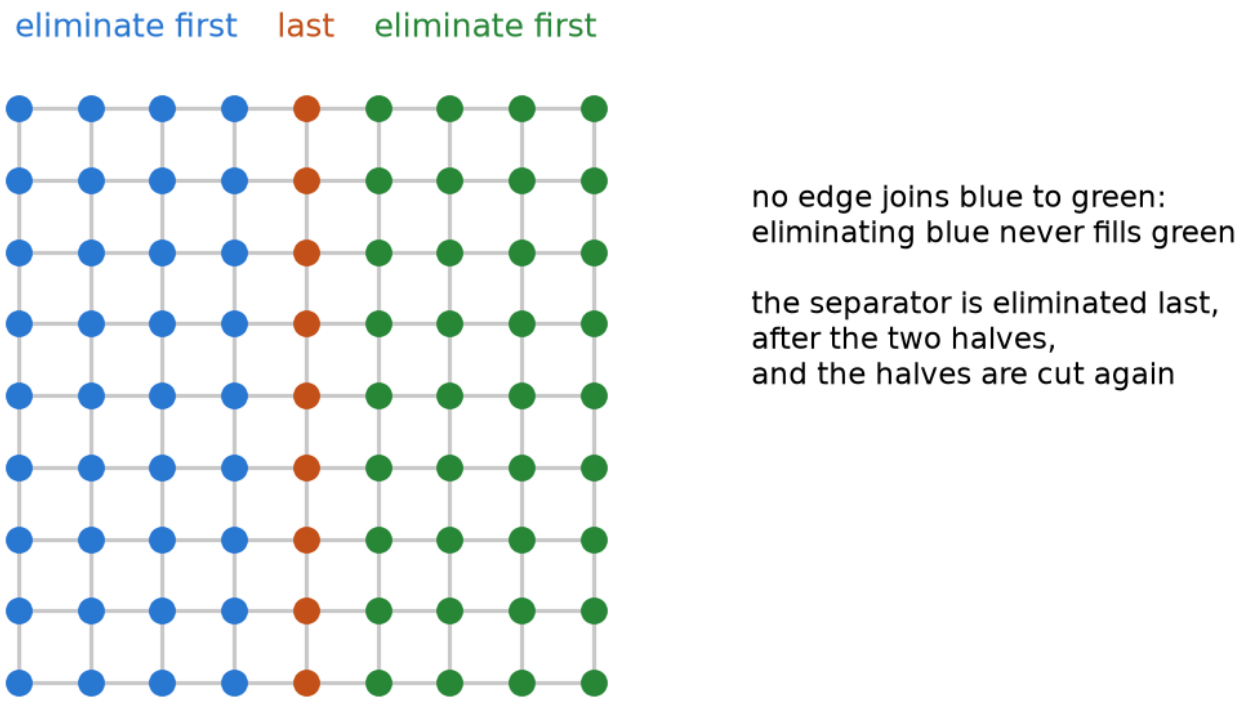
OUTPUT

```
the 60 x 60 grid: N = 3,600, 17,760 nonzeros in A
order of elimination             nonzeros of L + U  times those of A
natural order                                428,518              24.1
COLAMD (SuperLU's default)                   176,120               9.9
minimum degree                               104,830               5.9
```

OUTPUT

```
      grid        N             order   nonzeros of L + U  seconds to solve
   80 x 80      6,400           natural           1,024,158              0.031
   80 x 80      6,400            COLAMD             370,282              0.008
  160 x 160    25,600           natural           8,192,318              0.387
  160 x 160    25,600            COLAMD           2,036,562              0.049
```

Three orders. The *natural* order takes the unknowns as they come. *Minimum degree* eliminates at each step the node with the fewest remaining neighbours, since joining d neighbours creates at most $d(d-1)\,/\,2$ edges; it is greedy, and it can be beaten. *Nested dissection* cuts the graph by a **separator**, a set of nodes whose removal splits it in two, eliminates the two halves first and the separator last, and applies the same cut inside each half: no edge joins the halves, so eliminating one half can never fill the other, and the fill is confined to the separators. On a grid of N nodes this gives $O(N\log N)$ nonzeros in the factors and $O(N^{1.5})$ work, against $O(N^2)$ nonzeros in the natural order and $O(N^3)$ for dense elimination. Finding a good order is a problem about graphs, not about arithmetic.



Nested dissection: the separator, in orange, is eliminated last, after the two halves it keeps apart.

In the meeting.

The fourth tab of the solvers page, with the graph beside its matrix. The star first: its name guessed by the class; the first step of elimination replayed on it, the first row subtracted from every other, and the question what happens to the empty entries: "they're going to be filled". Roughly $3n$ nonzeros become about n^2 after one step, "everything's going to be dense here", and on a star of a million nodes that is out of memory on any computer, where the sparse system itself is solved in milliseconds. Then the pivot in the other sense: not the largest entry, chosen for stability, but the row with the fewest nonzeros, the minimum degree. On the graph, eliminating a node joins all its neighbours into a *clique*; eliminating a leaf of the star instead joins nothing, since it has one neighbour, and leaf by leaf the whole matrix was eliminated with no fill at all: "I can solve a matrix that has a billion nonzeros without creating sometimes no fill at all."

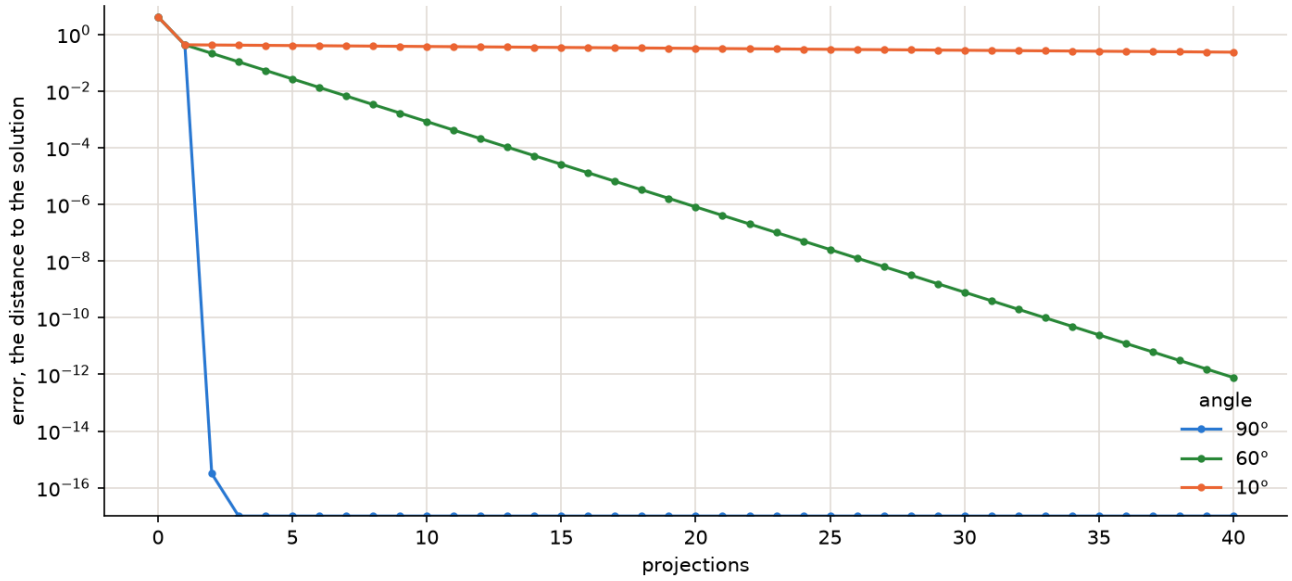
Then the walk's graph, the one from the start of the meeting: in the natural order, a node eliminated, red edges and red cells created, more with each node, "a very dense block" by the halfway point; in minimum degree, node after node and not one red cell, "this one is very lucky", a matrix for which the greedy order creates nothing. Then the 6×6 grid: natural order, red cells accumulating into a block that stays contained; minimum degree, a different pattern that looked no better and was, in the counts, better; and nested dissection, played twice to see the pattern: the nodes of one corner first, then the next corner, then the other two, and the middle row and column last, "it subdivides it into parts, and then works on its parts separately, from the smaller parts to the bigger parts, and at the very end globally at the middle". The page's counters read from the screen: the natural order 250 new nonzeros and 1,045 units of work; minimum degree 547; nested dissection 816, "and when you make it bigger, it is the best". The remark that followed: the instructor's doctoral advisor is one of the people who worked out how to find such orders, which is a problem in graph algorithms, the subject of a later course and of a part of this one.

Solving by iterating

Every method so far was **direct**: a fixed recipe that, with exact arithmetic, would end at the exact solution. An **iterative method** starts from a guess, any guess, and improves it by cheap steps until the residual is as small as asked: never exactly there, but as close as wanted, which is also what the simulations of the walk did in their own way. The simplest step is the **projection** of a point onto a line or a plane: the nearest point of it, at the foot of the perpendicular, which is what a shadow is. Kaczmarz's method, from 1937, projects the current guess onto the first equation, then onto the second, and around again:

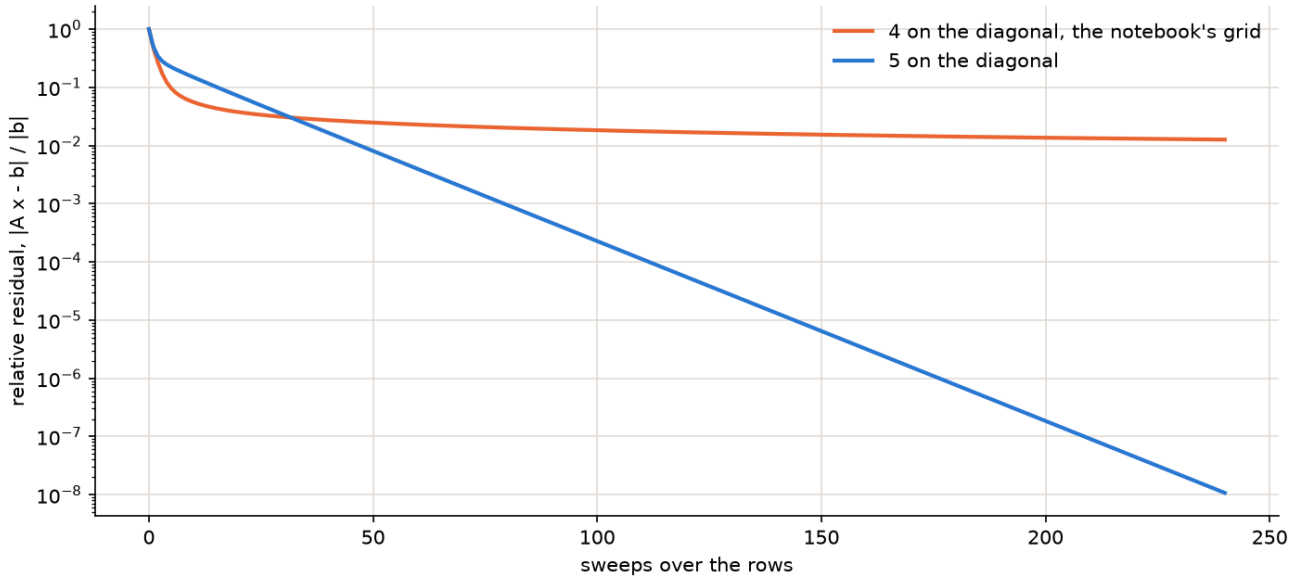
$$x \leftarrow x - \frac{a_i \cdot x - b_i}{a_i \cdot a_i}\, a_i \,.$$

After the step, equation i holds exactly and the others have been disturbed a little; each step costs one row, a dot product and an update, as many multiplications as the row has nonzeros. The guess zigzags toward the crossing. With two lines each cycle of two projections multiplies the error by $\cos^2\theta$, θ the angle between them: zero at a right angle, where two steps finish; a quarter at 60 degrees; 0.97 at 10 degrees, a thousand cycles for a factor of a billion. The angle is the conditioning seen from the iteration's side: an ill-conditioned system is slow to iterate for the same reason it was fragile to eliminate.



The error after each projection, at three angles: a fixed factor each cycle.

On a sparse system a sweep over all the rows costs the nonzeros, thousands against the millions of an elimination, and the number of sweeps is set by the conditioning. The deck's experiment runs sweeps on the 40×40 grid's system: with the notebook's 4 on the diagonal, a nearly singular system, the residual creeps and is still 1.3e-02 after 240 sweeps; with 5 on the diagonal, a well-conditioned one, it falls by the same factor every sweep and passes a millionth after 177. The stopping rule is a tolerance on the residual, the thing we can measure, and the error that comes with it is the condition number's business, as in section 2.3. Taking the rows at random, in proportion to their size, instead of in turn, gives a guarantee: the expected error shrinks by a fixed factor at every projection, one minus the inverse square of a condition number of A, the randomized Kaczmarz method of Strohmer and Vershynin, 2009. Kaczmarz is one of a family, with Jacobi and Gauss–Seidel, which solve each equation for its own unknown and go around, and conjugate gradients, `scipy.sparse.linalg.cg`, which needs far fewer sweeps on a symmetric system; all of them only read the matrix and never fill it.



Sweeps of Kaczmarz on the grid's system, nearly singular with 4 on the diagonal and well conditioned with 5.

Code: the zigzag at three angles, the error against the projections, and the sweeps on the grid

cells `dk-kacz-2d`, `dk-kacz-rate`, `dk-kacz-grid`

```
def kaczmarz_path(rows, x0, steps):
    """The points visited by Kaczmarz's method on the rows [a1, a2, b], cycling through them."""
    x = np.array(x0, dtype=float)
    path = [x.copy()]
    for k in range(steps):
        a, b = np.array(rows[k % len(rows)][:2]), rows[k % len(rows)][2]
        x = x - (a @ x - b) / (a @ a) * a
        path.append(x.copy())
    return np.array(path)


def two_lines(angle_deg):
    """x + y = 2, and a second line through (1, 1) at the angle, with the same norm."""
    th = np.pi / 4 + np.radians(angle_deg)
    r2 = np.sqrt(2) * np.array([np.cos(th), np.sin(th)])
    return [[1.0, 1.0, 2.0], [r2[0], r2[1], r2[0] + r2[1]]]


START = (4.2, 3.6)
ANGLES = [90, 60, 10]
fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.6))
for ax, deg in zip(axes, ANGLES):
    rows = two_lines(deg)
    xs = np.linspace(-1, 5, 2)
    for r, colour in zip(rows, ("#2a78d6", "#eb6834")):
        ax.plot(xs, (r[2] - r[0] * xs) / r[1], color=colour, linewidth=2.5)
    path = kaczmarz_path(rows, START, 12)
    ax.plot(path[:, 0], path[:, 1], "-", color="#1a1a1a", linewidth=1.4, alpha=0.8)
    ax.plot(path[:, 0], path[:, 1], "o", color="#1a1a1a", markersize=3.5)
    ax.plot(*START, "o", color="#999999", markersize=7)
    ax.plot(1, 1, "o", markerfacecolor="none", markeredgecolor="#CC0033", markersize=11,
```

```
markeredgewidth=2)
    err = np.hypot(path[-1, 0] - 1, path[-1, 1] - 1)
    ax.set_title(f"{deg}° between the lines: after 12 steps the error is {err:.2g}", fontsize=11)
    ax.set_xlim(-1, 5); ax.set_ylim(-1, 5); ax.set_aspect("equal"); ax.grid(True, color="#e1e0d9")
    ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()
```

```
fig, ax = plt.subplots(figsize=(9.5, 4.4))
print(f"{'angle':>7s}{'cos^2 of the angle':>20s}{'error after 2 steps':>21s}{'after 20':>10s}{'after
40':>10s}")
for deg, colour in zip(ANGLES, ("#2a78d6", "#2a8a3a", "#eb6834")):
    path = kaczmarz_path(two_lines(deg), START, 40)
    err = np.hypot(path[:, 0] - 1, path[:, 1] - 1)
    ax.semilogy(np.arange(len(err)), np.maximum(err, 1e-17), marker="o", markersize=3, color=colour,
label=f"{deg}°")
    print(f"{deg:6d}°{np.cos(np.radians(deg)) ** 2:20.3f}{err[2]:21.2g}{err[20]:10.2g}{err[40]:10.2g}")
ax.set_xlabel("projections"); ax.set_ylabel("error, the distance to the solution")
ax.set_ylim(1e-17, 10)
ax.grid(True, which="major", color="#e1e0d9"); ax.spines[["top", "right"]].set_visible(False);
ax.legend(frameon=False, title="angle")
fig.tight_layout()
plt.show()
```

OUTPUT

angle	cos^2 of the angle	error after 2 steps	after 20	after 40
90°	0.000	3.1e-16	0	0
60°	0.250	0.21	8.1e-07	7.7e-13
10°	0.970	0.42	0.32	0.23

```
def kaczmarz_sweeps(A_csr, b, sweeps):
    """Cyclic Kaczmarz on a CSR matrix from x = 0: the relative residual after each sweep."""
    x = np.zeros(A_csr.shape[0])
    data, indices, indptr = A_csr.data, A_csr.indices, A_csr.indptr
    norms = np.array([data[indptr[i]:indptr[i + 1]] @ data[indptr[i]:indptr[i + 1]] for i in
range(A_csr.shape[0])])
    history = [np.linalg.norm(A_csr @ x - b) / np.linalg.norm(b)]
    for _ in range(sweeps):
        for i in range(A_csr.shape[0]):
            r, idx = data[indptr[i]:indptr[i + 1]], indices[indptr[i]:indptr[i + 1]]
            x[idx] -= (r @ x[idx] - b[i]) / norms[i] * r
        history.append(np.linalg.norm(A_csr @ x - b) / np.linalg.norm(b))
    return np.array(history)


n_grid = 40
A4 = grid_matrix(n_grid).tocsr()                                # the notebook's grid: 4 on the diagonal
A5 = (A4 + sparse.identity(A4.shape[0], format="csr")).tocsr()    # the same, with 5: strictly diagonally
dominant
x_true = rng.random(A4.shape[0])
SWEEPS = 240
hist = {name: kaczmarz_sweeps(A_, A_ @ x_true, SWEEPS) for name, A_ in (("4 on the diagonal, the
notebook's grid", A4), ("5 on the diagonal", A5))}
N_ = A4.shape[0]
print(f"the {n_grid} x {n_grid} grid: N = {N_:,} unknowns, {A4.nnz:,} nonzeros; one sweep is {A4.nnz:,}
```

```
multiply-adds, "
      f"elimination in the natural order about {N_ * n_grid ** 2 // 1:,}")
for name, h in hist.items():
    below = np.argmax(h < 1e-6) if (h < 1e-6).any() else None
    print(f"{name:40s} residual after {SWEEPS} sweeps {h[-1]:.1e}" + (f"; below 1e-6 after {below} sweeps"
if below else "; not yet below 1e-6"))

fig, ax = plt.subplots(figsize=(9.5, 4.4))
for (name, h), colour in zip(hist.items(), ("#eb6834", "#2a78d6")):
    ax.semilogy(np.arange(len(h)), h, color=colour, linewidth=2, label=name)
ax.set_xlabel("sweeps over the rows"); ax.set_ylabel("relative residual, |A x - b| / |b|")
ax.grid(True, which="major", color="#e1e0d9"); ax.spines[["top", "right"]].set_visible(False);
ax.legend(frameon=False)
fig.tight_layout()
plt.show()
```

OUTPUT

```
the 40 x 40 grid: N = 1,600 unknowns, 7,840 nonzeros; one sweep is 7,840 multiply-adds,
elimination in the natural order about 2,560,000
4 on the diagonal, the notebook's grid   residual after 240 sweeps 1.3e-02; not yet below 1e-6
5 on the diagonal                        residual after 240 sweeps 1.1e-08; below 1e-6 after 177
sweeps
```

In the meeting.

The last part, on the fifth tab of the solvers page, began from a projection: a point above a plane, and its shadow on the plane, the nearest point, "you send light, and you get to the underlying plane at the shortest point". Then the lost point in the plane, far from the solution of two lines: one projection onto the blue line, the nearest point of it, at a right angle; then onto the orange one, closer; then the blue again, "I'm going to alternate where I'm projecting myself", and the residual and the error in the chart beside it fell together until the point could no longer be told from the solution and only the numbers still moved. "If you want to solve a linear system, you go over your equations and start projecting yourself to each equation one at a time." Then in space, three planes, the point projected onto the blue, the orange, the green, and around, the error dropping with every step while the box turned. The method was named iterative, "based on iterations that progressively move you closer to the point that you want to go, and make the error smaller a little bit at a time", against the direct methods, which with exact arithmetic end at the exact answer where an iteration never does, only arbitrarily close: paused at a residual of 10^{-16}, "this is fairly small, you cannot go below that", and on a larger system with worse conditioning the residual would stall near 10^{-8} instead, the loss of precision again. Last, the rows taken at random rather than in turn, "and that's even better", an algorithm still being analysed ten years ago; and the closing word on why exact solutions matter beside the guesses of AI: an engineer who solves a system better builds a plane at half the cost and half the weight, "and everything contributes to the progress".

After the meeting

Each student uploads their own copies of both notebooks to **Upload Your Class Work - Meeting 13**, graded on the eight cells marked **Your thinking** and **Your prompt**: four in the first notebook (one line for a system of any size; the pictures of the solutions; measured data and the condition number; the guess at the walk's equations) and four in the second (the constant factors in $O(n^3)$; a small residual and a large error; the prompt for `dense_to_csr`; choosing sparse storage). The group's two posts earn the attention mark for everyone they name; students who did not post earned it by signing the sheet in the meeting.

Keep your copies of the notebooks, with your measurements in them. **Review Quiz 13** on Canvas asks about them, open until Sunday, October 18: sixteen questions answered with a choice, on the matrix form, rank and singular systems, nearly parallel lines, the digits a condition number leaves, the residual, elimination and back substitution and their count, the inverse against `solve`, the arrays of CSR, the memory of a grid, where the sparse product wins, the equations for the walk, and fill-in on the star; and one question of a few sentences about your own condition numbers, solve times and crossover density. **Homework 1**, *Vectorization*, is due the same Sunday. The three pages of the meeting stay on the labs site and are linked above and on the meeting's page; the walk's reveal, the fill-in orders and the projections are there to replay. Meeting 14 names the matrix of a graph and takes up fill-in and the orders of elimination where this meeting left them.